

Queue — Fundamentals, BFS and the Queue Patterns

What this is. A complete study guide for the **Queue** data structure, built from a guided reasoning-first session (fundamentals, amortized cost, ten queue patterns, templates and ten hard problems) and then **extended to cover everything the session only named**: implementations from scratch, every problem the session listed but did not solve, tree and graph BFS variants, grid problems, the monotonic deque family, Kahn's algorithm with cycle detection, 0–1 BFS, and a decision guide.

The one idea to keep. Ask yourself **"What determines who is processed next?"** If the answer is *the order in which they arrived* → **queue**. If it is *the distance from where I started* → **BFS** (a queue again). If it is *a priority* → **heap**. The hardest step is almost always deciding **what one element of the queue represents** — once that is right, the code is mechanical.

What the original session covered — and what this guide adds

Topic	Original session	This guide
Queue = FIFO, enqueue / dequeue / peek, deque, why <code>list.pop(0)</code> is $O(n)$, queue vs stack, queue vs heap	✓	✓ (+ diagrams, implementations from scratch: array + head pointer, linked list, ring buffer, two stacks)
Amortized time complexity (the "each element enters once, leaves once" argument)	✓	✓ (+ the counter-example that is <i>not</i> amortized $O(1)$)
Pattern 1 · FIFO simulation — Time Needed to Buy Tickets, Number of Recent Calls, Moving Average	✓ (reasoning, two with code)	✓ full solutions, traces, $O(n)$ formula
Number of Students Unable to Eat Lunch, Dota2 Senate, Reveal Cards in Increasing Order	⚠ named (Dota2 solved)	✓ all solved
Pattern 2 · BFS — level order, <code>level_size</code> , shortest path in an unweighted graph, BFS vs DFS	✓ (concepts)	✓ (+ code, path reconstruction, Right Side View, Zigzag, Min Depth, Word Ladder, Open the Lock, Perfect Squares, Snakes and Ladders)
Pattern 3 · Multi-source BFS	✓ (concept, 01 Matrix)	✓ (+ Rotting Oranges, Walls and Gates, As Far from Land as Possible)
Pattern 4 · BFS + state, visited over the <i>whole</i> state, K walls	✓ (wall break, code with a bug)	✓ corrected code (+ Obstacles Elimination, All Keys with a bitmask, Minimum Knight Moves)
Pattern 5 · BFS on grids, mark-on-enqueue, DFS vs BFS rule	✓ (concepts)	✓ (+ Binary Matrix, Nearest Exit, Islands, Escape a Large Maze)
Pattern 6 · Monotonic deque, two removal rules, indices	✓ (concept, Constrained Subsequence Sum)	✓ (+ Sliding Window Max/Min, Shortest Subarray with Sum $\geq K$, Abs-Diff Limit, Jump Game VI)
Pattern 7 · Queue + hash map	✓ (first non-repeating)	✓ (+ First Unique Character, First Unique Number)
Pattern 8 · Queue simulation (round robin)	✓	✓ (+ timeline and waiting-time trace)

Topic	Original session	This guide
Pattern 9 · Topological sort (Kahn), cycle detection, "disconnected $\neq$ cyclic"	✓ (+ Alien Dictionary)	✓ (+ Course Schedule I/II, Parallel Courses)
Pattern 10 · 0–1 BFS, front/back, relaxation	✓ (+ Min Cost Valid Path)	✓ (+ Minimum Obstacle Removal, proof of the two-distance invariant)
Templates, recognition table, "what determines next" map	✓	✓ (+ pitfalls, complexity table, practice list, interview Q&A)

More than 60 problems and implementations are solved in Python. Every program was run against a brute-force or independent solution on thousands of random inputs, and every step table in this guide was produced by running the algorithm.

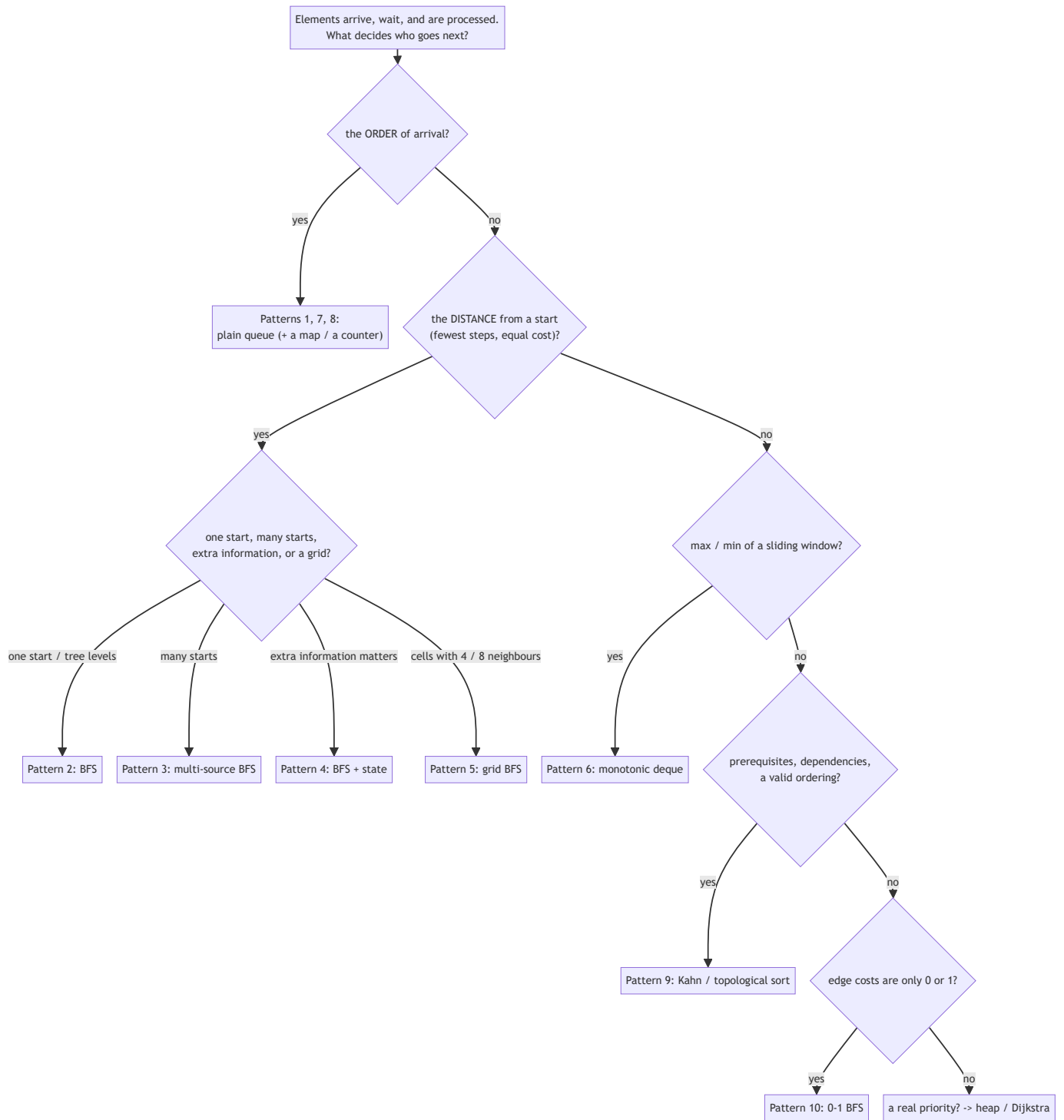
Corrections to the original session. Five points needed fixing, each marked **Correction** where it occurs: (1) the *break one wall* BFS enqueued the wall state **without** checking `visited` (duplicates in the queue) — see §5.3; (2) the 0–1 BFS picture `[5 5 5 6 6 7 7]` is impossible: the deque holds **at most two distinct distances, d and $d+1$** — §11.3; (3) Word Ladder's complexity is $O(26 \cdot N \cdot L^2)$, not $O(26 \cdot N \cdot L)$, because building each candidate word costs $O(L)$ — §3.7; (4) the Time Needed to Buy Tickets code **mutates its input** and an $O(n)$ formula exists — §2.1; (5) "Task Scheduler-style simulations" is listed under *queue simulation*, but Task Scheduler (LeetCode 621) is a **greedy / heap-counting** problem (solved in the Heap guide) — §9.2.

Part 0 · Roadmap

0.1 The queue patterns at a glance

#	Pattern	What does one queue element represent?	Typical problems
1	FIFO simulation	someone / something waiting for its turn	Time Needed to Buy Tickets, Recent Calls, Moving Average, Students & Sandwiches, Dota2 Senate, Reveal Cards
2	BFS / level order	a node waiting to be explored, nearer nodes first	Level Order, Right Side View, Word Ladder, Open the Lock, Snakes and Ladders
3	Multi-source BFS	a source that expands together with all the others	Rotting Oranges, 01 Matrix, Walls and Gates, As Far from Land
4	BFS + state	a <i>(position, extra information)</i> pair	Obstacles Elimination, Shortest Path to Get All Keys, Knight Moves
5	BFS on grids	a cell (or a cell + something)	Shortest Path in Binary Matrix, Nearest Exit, Number of Islands, Escape a Large Maze
6	Monotonic deque	an index that could still become the window's best	Sliding Window Maximum, Shortest Subarray with Sum $\geq K$, Constrained Subsequence Sum
7	Queue + hash map	an arrival whose <i>validity</i> the map decides	First Non-Repeating Character in a Stream, First Unique Number
8	Queue simulation	a task that gets a time slice and may return	Round Robin, Time Needed to Buy Tickets
9	Topological sort (Kahn)	a node whose prerequisites are all done	Course Schedule I / II, Alien Dictionary, Parallel Courses
10	0-1 BFS	a node reached at distance d or $d+1$	Min Cost to Make a Valid Path, Minimum Obstacle Removal

0.2 First-glance decision tree

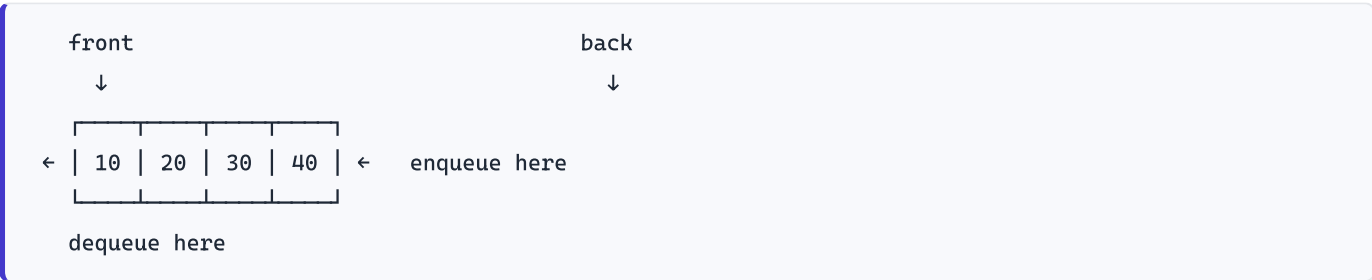


Part 1 · Queue Fundamentals

The important thing is not learning `enqueue()` and `dequeue()`. The important question is: why would an algorithm want to process things in the order they arrived?*

1.1 What is a queue?

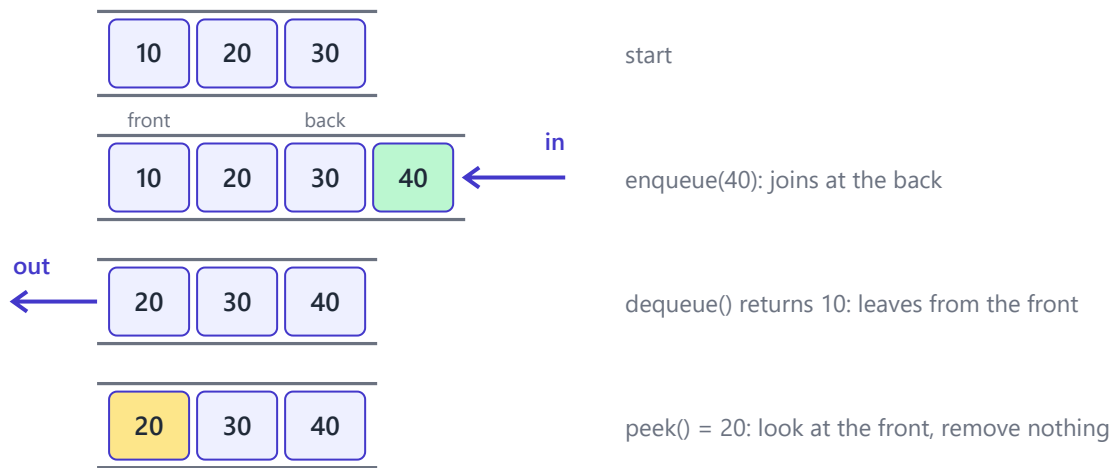
A queue is a data structure where **items enter at one end (the back / rear) and leave from the other (the front)**. Think of people waiting in a line: the person who arrived first is served first.



This is **FIFO — First In, First Out**. Enqueue `10, 20, 30`; the removal order is `10 → 20 → 30`.

1.2 The fundamental operations

The queue operations: enqueue at the back, dequeue at the front



Operation	Meaning	Python deque	Cost
enqueue	add at the back	<code>q.append(x)</code>	$O(1)$
dequeue	remove and return the front	<code>q.popleft()</code>	$O(1)$
peek / front	look at the front without removing	<code>q[0]</code>	$O(1)$
is_empty	any items?	<code>not q</code>	$O(1)$
size	how many items	<code>len(q)</code>	$O(1)$

- **Underflow** — dequeuing or peeking an empty queue (`IndexError` in Python). Guard with `while q:` / `if q:`.

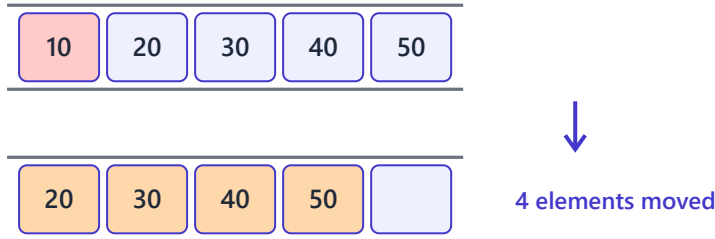
- **Overflow** — enqueueing into a full *fixed-capacity* queue (see the ring buffer in §1.6). A Python `deque` grows dynamically unless you give it `maxlen`.

1.3 Why not a normal Python list?

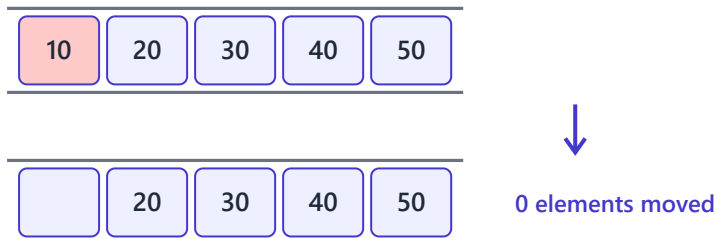
```
q = [10, 20, 30, 40]
q.pop(0)      # removes 10 ... but every remaining element has to shift left
```

Why `q.pop(0)` on a list is $O(n)$ but `deque.popleft()` is $O(1)$

list: remove the front, then EVERY other element shifts one slot left



deque: only the head pointer moves - nothing is copied



`list.pop(0)` (and `list.insert(0, x)`) are $O(n)$ because the elements after the removed one all move. A `deque` keeps both ends cheap:

```
from collections import deque
q = deque([10, 20, 30, 40])
q.popleft()      # O(1)
q.append(50)     # O(1)
```

For DSA problems a `collections.deque` is almost always the right queue. (A list is fine for a *stack*, not for a queue.)

1.4 The deque: a double-ended queue

A **deque** ("deck") allows insertion and removal at **both** ends in $O(1)$:

Operation	Method	Cost
add at back / front	<code>append(x)</code> / <code>appendleft(x)</code>	$O(1)$
remove from back / front	<code>pop()</code> / <code>popleft()</code>	$O(1)$

Operation	Method	Cost
look at back / front	<code>d[-1] / d[0]</code>	$O(1)$
access the middle <code>d[i]</code>	<code>d[i]</code>	$O(n)$ — a deque is not built for random access
rotate	<code>d.rotate(k)</code>	$O(k)$
bounded	<code>deque(maxlen=k)</code>	the oldest item falls off automatically

A plain queue uses one end for each job. The **same structure** with both ends used is the tool for the *monotonic deque* (Pattern 6) and *0–1 BFS* (Pattern 10).

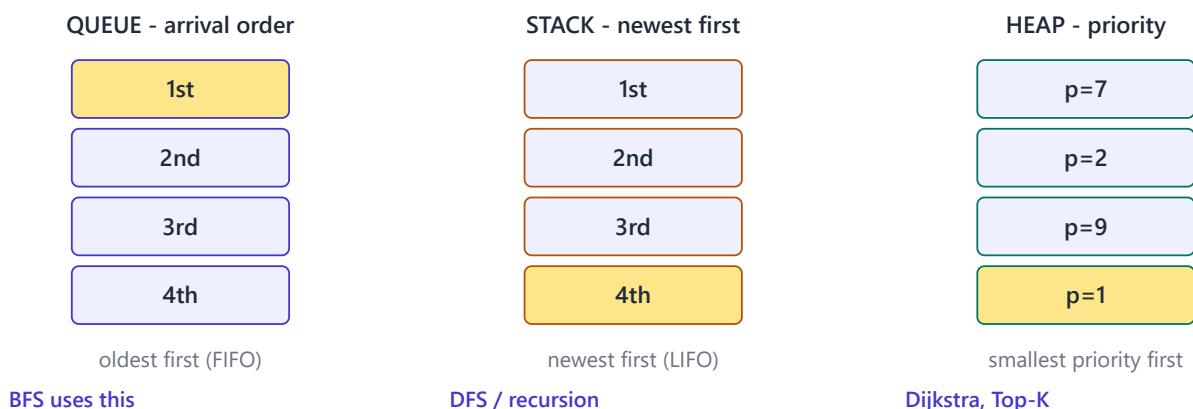
1.5 Queue vs Stack vs Deque vs Heap

Think about **who is served next**:

QUEUE 10 ← first 20 30 remove 10 ↓ FIFO	STACK 10 20 30 ← last remove 30 ↓ LIFO
---	--

Structure	Next element is...	Typical use	Insert / remove
Queue	the oldest	BFS, scheduling, streams	$O(1)$ / $O(1)$
Stack	the newest	DFS, undo, brackets, monotonic stack	$O(1)$ / $O(1)$
Deque	oldest <i>or</i> newest (your choice)	sliding-window extremes, 0–1 BFS	$O(1)$ both ends
Heap	the best priority — arrival order does not matter	Top-K, Dijkstra, scheduling by priority	$O(\log n)$

Who gets processed next? The rule decides the data structure



- **BFS** → **queue** → **explore the oldest waiting state first.**
- **DFS** → **stack / recursion** → **explore the most recently discovered state first.**
- A heap is different from both: it doesn't care about arrival order, it chooses by priority.

1.6 Representing a queue in Python

Option	Verdict
<code>collections.deque</code>	the default — $O(1)$ at both ends
<code>list</code> with <code>pop(0)</code>	works, but each dequeue is $O(n)$ — only for tiny inputs
<code>list</code> + a moving head index	$O(1)$ — but you must trim the dead prefix (below)
<code>queue.Queue</code>	thread-safe with blocking <code>put</code> / <code>get</code> ; slower — use it for producer/consumer threads, not for BFS
<code>asyncio.Queue</code>	the same idea for coroutines
<code>heapq</code>	a <i>priority</i> queue — a different structure (see the Heap guide)

Implementing a queue from scratch

1. A list with a head pointer. Never shift anything; just advance `head` . Trim the dead prefix once it dominates so memory stays bounded.

```

class ArrayQueue:
    """A queue on a Python list with a moving HEAD index: dequeue never shifts anything.
    The dead prefix is trimmed once it is more than half of the list, so the cost stays amortized
    O(1)."""

    def __init__(self):
        self._data = []
        self._head = 0

    def enqueue(self, x):
        self._data.append(x)

    def dequeue(self):
        if self._head == len(self._data):
            raise IndexError("dequeue from empty queue")           # queue UNDERFLOW
        x = self._data[self._head]
        self._head += 1
        if self._head * 2 > len(self._data):                         # more than half is dead space ->
compact
            self._data = self._data[self._head:]
            self._head = 0
        return x

    def peek(self):
        if self._head == len(self._data):
            raise IndexError("peek from empty queue")
        return self._data[self._head]

    def is_empty(self):
        return self._head == len(self._data)

    def __len__(self):
        return len(self._data) - self._head

```

2. A linked list. Keep **both** ends: enqueue at the *tail*, dequeue at the *head*. Every operation is $O(1)$ worst case. (Dequeuing the last node must also reset `tail`.)

```

class _Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.next = nxt

class LinkedQueue:
    """A queue on a singly linked list: keep BOTH ends. Enqueue at the tail, dequeue at the head:
    O(1) worst case."""

    def __init__(self):
        self._head = self._tail = None
        self._size = 0

    def enqueue(self, x):
        node = _Node(x)
        if self._tail is None:                # empty queue: the node is both head
and tail
            self._head = self._tail = node
        else:
            self._tail.next = node
            self._tail = node
        self._size += 1

    def dequeue(self):
        if self._head is None:
            raise IndexError("dequeue from empty queue")
        node = self._head
        self._head = node.next
        if self._head is None:                # we removed the last node
            self._tail = None
        self._size -= 1
        return node.val

    def peek(self):
        if self._head is None:
            raise IndexError("peek from empty queue")
        return self._head.val

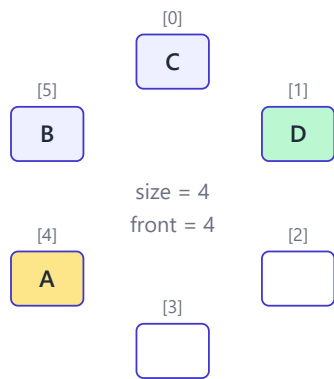
    def is_empty(self):
        return self._head is None

    def __len__(self):
        return self._size

```

3. A ring buffer (circular queue) — LeetCode 622. A fixed capacity `k`; indices wrap with `% k`. Keep `front` and `size` (rather than head/tail) and you never confuse "full" with "empty".

MyCircularQueue: a ring buffer (capacity 6, front = 4, size = 4)



- front slot = index 4 (holds A, the next to leave)
- rear slot = $(\text{front} + \text{size} - 1) \% \text{cap} = (4 + 4 - 1) \% 6 = 1$ (D)
- next free = $(\text{front} + \text{size}) \% \text{cap} = 8 \% 6 = 2$
- dequeue : $\text{front} = (\text{front} + 1) \% \text{cap}$
- full when $\text{size} == \text{cap}$; empty when $\text{size} == 0$

```

class MyCircularQueue:
    """LeetCode 622 - a fixed-capacity queue in a ring buffer. `front` is the index of the head
    element,
    `size` how many elements are stored; the tail slot is (front + size) % capacity."""

    def __init__(self, k):
        self.buf = [0] * k
        self.front = 0
        self.size = 0
        self.cap = k

    def enQueue(self, value):
        if self.isFull():
            return False
        self.buf[(self.front + self.size) % self.cap] = value
        self.size += 1
        return True

    def deQueue(self):
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.cap
        self.size -= 1
        return True

    def Front(self):
        return -1 if self.isEmpty() else self.buf[self.front]

    def Rear(self):
        return -1 if self.isEmpty() else self.buf[(self.front + self.size - 1) % self.cap]

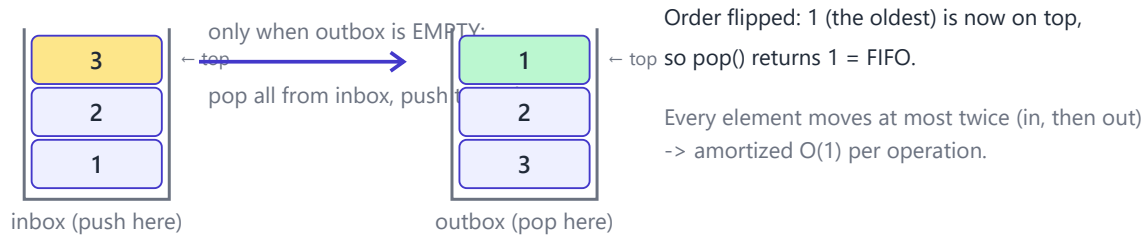
    def isEmpty(self):
        return self.size == 0

    def isFull(self):
        return self.size == self.cap

```

4. A queue from two stacks — LeetCode 232. `inbox` receives pushes; `outbox` serves pops; when `outbox` is empty, pour `inbox` into it (which reverses the order).

Queue from two stacks: inbox for push, outbox for pop



```
class MyQueue:
    """LeetCode 232 - a queue from two stacks. `inbox` receives pushes, `outbox` serves pops; when
    `outbox` runs dry we pour the whole inbox into it (which reverses the order). Every element moves at most
    once."""

    def __init__(self):
        self.inbox, self.outbox = [], []

    def push(self, x):
        self.inbox.append(x)

    def _fill(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())

    def pop(self):
        self._fill()
        return self.outbox.pop()

    def peek(self):
        self._fill()
        return self.outbox[-1]

    def empty(self):
        return not self.inbox and not self.outbox
```

Each element is moved from `inbox` to `outbox` **at most once**, so `pop` is $O(1)$ **amortized** even though one `pop` can move many elements.

1.7 Amortized time complexity

An individual operation may occasionally be expensive, but if we look at a **whole sequence** of operations, the average cost per operation can still be small.

It is *not* "average the Big-O of each call". It is: **spread the occasional expensive work over all the operations that caused it.**

Take *Number of Recent Calls* (§2.2). `append` and `popleft` are $O(1)$, but

```
while window[0] < t - 3000:
    window.popleft()
```

can remove *many* timestamps in one call — that single call costs $O(n)$. Yet:

Amortized cost: every timestamp enters once and leaves once

ping #1	ping #2	ping #3	ping #4	ping #5	ping #6
+1 append	+1 append	+1 append	+1 append	+1 append	+1 append
no pops	no pops	no pops	-3 popleft	no pops	-2 popleft

one call may pop many (ping #4 pops 3), but the total number of pops can never exceed the total number of appends
6 appends + at most 6 pops = at most 12 operations for 6 calls -> $O(1)$ amortized per call

- every timestamp is **appended once** and **removed at most once**,
- so n calls perform at most n appends and n removals — **$O(n)$ in total**, i.e. **$O(1)$ amortized per call**.

A shelf analogy: you can never take 50 books off a shelf you have not first put 50 books on. The total number of removals is capped by the total number of additions.

	Case A — repeated work	Case B — permanent removals
Operation i touches	i elements (the <i>same</i> ones again)	elements that are removed forever
Total	$1 + 2 + \dots + n = O(n^2)$	at most n removals = $O(n)$
Amortized per op	$O(n)$ — not $O(1)$	$O(1)$

The question to ask when you see a `while` inside a loop:

Can the same element be processed repeatedly, or only a limited number of times overall? If each element **enters once and leaves once**, the inner `while` is amortized $O(1)$ per element. This argument returns for **monotonic dequeues** (§7), **queues with stale entries** (§8) and **Kahn's algorithm** (§10).

1.8 The core queue mental model

things arrive in an order $\rightarrow$ [front ... back] $\rightarrow$ things are processed in that same order

$\uparrow$
 unfinished ones (go back to the rear)

The **invariant** to write down for every queue problem:

"The front of the queue is always the element whose turn is next, and the queue contains exactly the elements that are still waiting."

What "waiting" means changes with the pattern — people who still need tickets, nodes not yet explored, tasks not yet finished, courses whose prerequisites are done — but the sentence stays the same.

1.9 The fundamental recognition idea

Don't memorise "*Rotting Oranges* = *queue*". Ask:

1. **What is one element of my queue?** (a person, a node, a cell, a *(cell, state)* pair, an index ...)
2. **What determines who is next?** (arrival order, distance, ...)
3. **What enters the queue, and when do I mark it as seen?**
4. **What is the invariant that makes the answer correct?**

1.10 What can go wrong — quick facts

Mistake	Symptom / fix
<code>list.pop(0)</code> in a loop	correct but $O(n^2)$ — use <code>deque.popleft()</code>
Marking visited when you dequeue instead of when you enqueue	the same state enters the queue many times (§6.2)
Forgetting <code>visited</code> on a graph	infinite loop
<code>for _ in range(len(q))</code> written as <code>while len(q)</code> inside a level loop	the level boundary is lost — freeze the size before the loop
Storing only <code>(row, col)</code> when extra information changes your options	wrong answers — add it to the state (§5)
Indexing <code>d[i]</code> in the middle of a deque	$O(n)$ per access
Using the <i>answer variable</i> instead of the <i>queue front</i> as the "current node"	skips nodes
Kahn: starting from one node instead of all indegree-0 nodes	misses other components

Part 2 · Pattern 1 — Basic FIFO Simulation

Recognition. *Things arrive in an order and must be processed in that same order; if something is not finished it goes back to the rear.* The question to ask: **"What determines who gets processed next?"** If the answer is *the order in which they are waiting*, think **FIFO queue**.

```
from collections import deque
q = deque(initial_items)
while q:
    item = q.popleft()          # FRONT
    # ... process item ...
    if not_finished:
        q.append(item)         # BACK
```

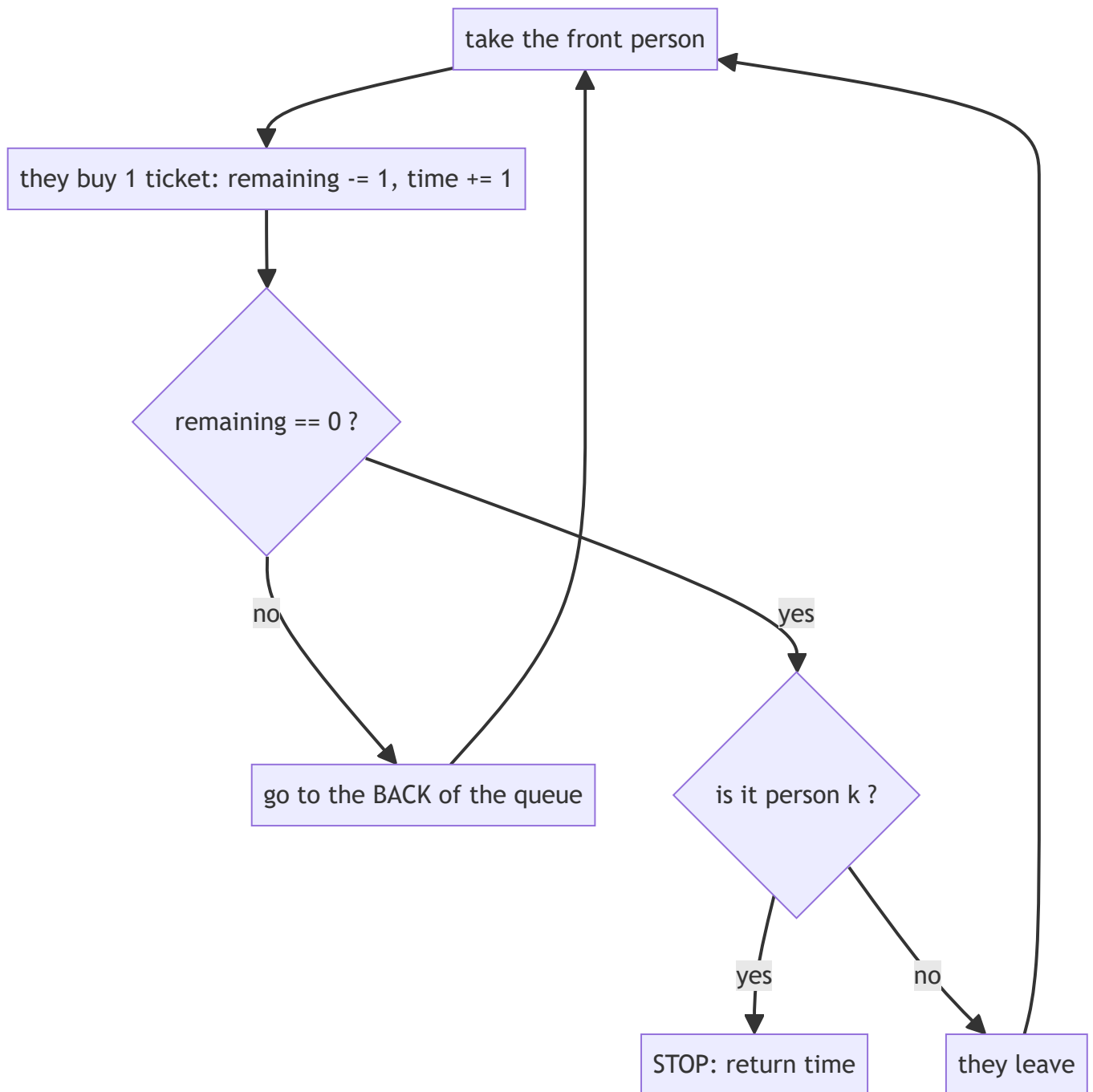
Mental model: **FRONT** → **process** → **BACK** (if unfinished).

2.1 Time Needed to Buy Tickets (LeetCode 2073)

n people stand in a queue; `tickets[i]` is how many tickets person i wants. Every second the person at the front buys **one** ticket; if they still need more they go to the **back**, otherwise they leave. Return the number of seconds until person k has bought all their tickets. `tickets = [2, 3, 2]`, $k = 2 \rightarrow 6$.

Derive it (answer these before coding):

1. *What determines who acts next?* The order in the queue.
2. *After buying one ticket, where does a person who still needs tickets go?* To the back.
3. *What data structure fits?* A queue.
4. *What is one element of the queue?* **A person** (their index).



Invariant. The queue contains exactly the people who still need tickets, in the order in which they should receive their next ticket. We stop **when person k reaches 0**, not earlier.

second	front	what happens	queue after (front → back)
0	-	-	[P0:2, P1:3, P2:2]
1	P0	P0 buys 1, needs 1 more, goes to the back	[P1:3, P2:2, P0:1]
2	P1	P1 buys 1, needs 2 more, goes to the back	[P2:2, P0:1, P1:2]
3	P2	P2 buys 1, needs 1 more, goes to the back	[P0:1, P1:2, P2:1]
4	P0	P0 buys the last one and leaves	[P1:2, P2:1]
5	P1	P1 buys 1, needs 1 more, goes to the back	[P2:1, P1:1]
6	P2	P2 buys the last one and leaves <- target k: STOP	[P1:1]

Answer: **6 seconds**.

The queue stores the **person index**, not the ticket count — the count lives in `tickets` (here a copy, so the input is not modified).

```
from collections import deque

def time_required_to_buy(tickets, k):
    """LeetCode 2073. The queue holds PERSON INDICES; the remaining tickets live in a copy of
    `tickets`."""
    remaining = list(tickets)
    q = deque(range(len(remaining)))
    time = 0
    while q:
        person = q.popleft()
        remaining[person] -= 1          # one second: they buy one ticket
        time += 1
        if remaining[person] > 0:
            q.append(person)           # still needs tickets: back of the line
        elif person == k:
            return time                # the target just finished
```

Complexity. Time $O(\text{total tickets purchased})$ — up to $O(n \cdot \max(\text{tickets}))$; space $O(n)$.

Correction. The original code decremented `tickets[person]` directly, **mutating the caller's list**. Copying first (`remaining = list(tickets)`) is a one-line fix.

The $O(n)$ shortcut. Nobody has to be simulated: person `i` buys tickets *only until* `k` finishes.

- People **at or before** `k` ($i \leq k$) buy `min(tickets[i], tickets[k])` — they go before `k` in every round.
- People **after** `k` ($i > k$) buy `min(tickets[i], tickets[k] - 1)` — in the final round `k` finishes before they get to buy.

```
def time_required_formula(tickets, k):
    """The  $O(n)$  shortcut: nobody ahead of k (or k itself) buys more than tickets[k] tickets,
    and nobody behind k buys more than tickets[k] - 1 before k finishes."""
    t = tickets[k]
    return sum(min(x, t) if i <= k else min(x, t - 1) for i, x in enumerate(tickets))
```

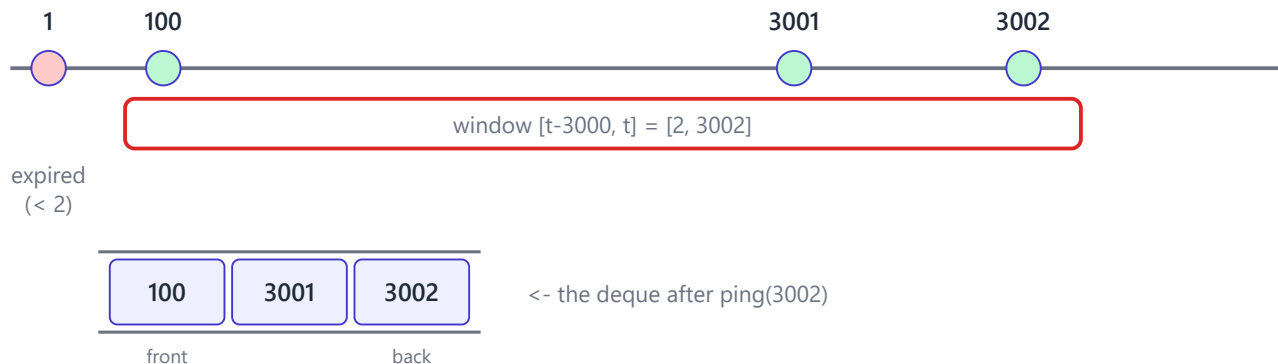
Recognition signal. Entities repeatedly take their turn and unfinished entities go to the back → FIFO queue. (*It also fits Pattern 8, queue simulation.*)

2.2 Number of Recent Calls (LeetCode 933)

Implement `ping(t)`: timestamps arrive in **strictly increasing** order; return how many calls happened in the inclusive interval `[t - 3000, t]`. `ping(1) → 1`, `ping(100) → 2`, `ping(3001) → 3`, `ping(3002) → 3`.

Notice *what kind of question this is*: it is not "process in arrival order" — it is "**which earlier calls are still inside the window?**"

Number of Recent Calls: ping(3002) keeps only timestamps in [2, 3002]



Derive it.

- When a call becomes too old, where is it relative to the newer calls? At the **front** (the oldest end).
- Which structure lets us add the newest at the back and remove old ones from the front? A **deque**.
- **Invariant:** the deque contains exactly the timestamps inside $[t - 3000, t]$.
- When is x too old? $x < t - 3000$.
- Why is it safe to check only the front? Timestamps are increasing, so if the front is not too old, **nothing behind it is either**.

ping(t)	window	popped from front	deque after	returned
1	$[-2999, 1]$	-	[1]	1
100	$[-2900, 100]$	-	[1, 100]	2
3001	[1, 3001]	-	[1, 100, 3001]	3
3002	[2, 3002]	1	[100, 3001, 3002]	3

```

from collections import deque

class RecentCounter:
    """LeetCode 933. The deque holds exactly the timestamps inside  $[t - 3000, t]$ ."""

    def __init__(self):
        self.window = deque()

    def ping(self, t):
        self.window.append(t)                    # newest goes to the back
        while self.window[0] < t - 3000:         # too old? then everything before it is too
old as well
            self.window.popleft()
        return len(self.window)

```

Complexity. Each timestamp is appended once and popped at most once → **$O(1)$ amortized per call, $O(n)$ for n calls** (see §1.7). Space $O(\text{window size})$.

2.3 Moving Average from Data Stream (LeetCode 346, premium)

Numbers arrive one at a time; `next(val)` returns the average of the last `size` numbers. `size = 3`: `next(1) = 1`, `next(10) = 5.5`, `next(3) = 4.67`, `next(5) = 6`.

Derive it.

- *Where does a new number go?* The back. *Which one disappears when the window is full?* The front (the oldest). *Which structure?* A queue.
- *Do we recompute the sum every time?* **No** — when one number enters and one leaves the sum changes by `+new - old`.

So we maintain **two things**:

Queue → remembers WHICH numbers are in the window (order)
Sum → remembers their TOTAL (information)

Invariant. The queue contains exactly the most recent `size` elements, and `sum` is the sum of everything in the queue.

next(val)	leaves the window	queue	sum	average
1	-	[1]	1	1/1 = 1
10	-	[1, 10]	11	11/2 = 5.5
3	-	[1, 10, 3]	14	14/3 = 4.67
5	1	[10, 3, 5]	18	18/3 = 6

```
from collections import deque

class MovingAverage:
    """LeetCode 346. The queue remembers WHICH numbers are in the window, 'total' remembers their sum."""

    def __init__(self, size):
        self.size = size
        self.q = deque()
        self.total = 0

    def next(self, val):
        self.q.append(val)
        self.total += val
        if len(self.q) > self.size:
            self.total -= self.q.popleft()          # the oldest number leaves the window
        return self.total / len(self.q)
```

Complexity. $O(1)$ per call. Why a queue and not a list? `pop(0)` is $O(n)$.

The broader pattern — "Queue handles ORDER, extra state handles INFORMATION". Here the extra state is a sum; in §8 it will be a hash map of frequencies.

Recognition signal. You continuously receive elements and only care about the most recent $k \rightarrow$ queue / deque + maintained state.

2.4 Number of Students Unable to Eat Lunch (LeetCode 1700)

Students stand in a queue, each preferring a circular (0) or a square (1) sandwich. A stack of sandwiches lies beside them. Repeat: if the **front** student prefers the **top** sandwich they take it and leave; otherwise they go to the **back**. Stop when nobody in the queue wants the top sandwich. Return the number of students left.

`students = [1,1,0,0]`, `sandwiches = [0,1,0,1]` $\rightarrow$ 0.

Derive it. What does the queue represent? The students still waiting, in order. What does the sandwich list represent? A **stack** — only the top matters, and it only changes when someone takes it. When does the process stop? When the queue makes a **full lap without anyone taking the top sandwich** — i.e. `misses == len(queue)`.

front student	top sandwich	action	queue after
1	0	student 1 refuses sandwich 0 $\rightarrow$ back (misses = 1)	[1, 0, 0, 1]
1	0	student 1 refuses sandwich 0 $\rightarrow$ back (misses = 2)	[0, 0, 1, 1]
0	0	student 0 takes sandwich 0 and leaves	[0, 1, 1]
0	1	student 0 refuses sandwich 1 $\rightarrow$ back (misses = 1)	[1, 1, 0]
1	1	student 1 takes sandwich 1 and leaves	[1, 0]
1	0	student 1 refuses sandwich 0 $\rightarrow$ back (misses = 1)	[0, 1]
0	0	student 0 takes sandwich 0 and leaves	[1]
1	1	student 1 takes sandwich 1 and leaves	[]

No more moves: **0 student(s)** cannot eat.

Getting stuck looks like this:

front student	top sandwich	action	queue after
1	1	takes it	[1, 1, 0, 0, 1]
1	0	refuses (misses = 1)	[1, 0, 0, 1, 1]
1	0	refuses (misses = 2)	[0, 0, 1, 1, 1]
0	0	takes it	[0, 1, 1, 1]
0	0	takes it	[1, 1, 1]
1	0	refuses (misses = 1)	[1, 1, 1]
1	0	refuses (misses = 2)	[1, 1, 1]
1	0	refuses (misses = 3)	[1, 1, 1]

The top sandwich is 0 but only 1s are left: a full lap of misses ends the process with **3 students** unfed.

```

from collections import deque

def count_students(students, sandwiches):
    """LeetCode 1700 (simulation version). Stop when a full lap of the queue finds nobody who wants
    the top sandwich."""
    q = deque(students)
    i = 0 # index of the sandwich on top of the stack
    misses = 0 # students in a row who refused the top
    sandwich
    while q and misses < len(q):
        if q[0] == sandwiches[i]:
            q.popleft()
            i += 1
            misses = 0
        else:
            q.append(q.popleft())
            misses += 1
    return len(q)

```

Complexity. $O(n^2)$ worst case (each sandwich may cost a lap). Space $O(n)$.

The insight that removes the queue. *The order of the students is irrelevant.* A sandwich blocks everyone **iff no remaining student wants it**, and that depends only on **how many** students want each type:

```

from collections import Counter

def count_students_counter(students, sandwiches):
    """LeetCode 1700 (counting version): the ORDER of the queue does not matter, only how many want
    each type."""
    want = Counter(students)
    for s in sandwiches:
        if want[s] == 0: # nobody left wants this sandwich: everyone
            behind it starves too
            break
        want[s] -= 1
    return sum(want.values())

```

$O(n)$ time. Whenever a simulation's outcome depends only on **counts**, ask whether the queue is needed at all.

2.5 Dota2 Senate (LeetCode 649)

The senate is a string of **R** (Radiant) and **D** (Dire). In each round, every senator in order may **ban** one senator of the opposing party (they lose all future turns), or, if all remaining senators are of one party, announce victory. Return the winning party. "RDD" → "Dire".

The difficulty is **not** the queue — it is representing "when is this senator's next turn?" after bans.

Derive it.

- Senators who act early can ban the *next* opponent that would otherwise act. The best ban is always **the opponent whose turn comes soonest**.
- Instead of storing letters, store each senator's **turn number** in a per-party queue: `Radiant = [0]`, `Dire = [1, 2]`.
- Compare the fronts. **The smaller number acts first**, bans the other front (which is simply dropped), and — because it survives — **votes again next round**, at turn $r + n$.

Invariant. Each queue holds the future turn numbers of the senators who are still alive; the smaller front acts first.

step	compare fronts	Radiant queue	Dire queue
start	-	[0]	[1, 2]
1	$R_0 < D_1$: R acts first, bans that D, R returns as 3	[3]	[2]
2	$D_2 < R_3$: D acts first, bans that R, D returns as 5	[]	[5]

Dire's queue is the only one left → **Dire** wins.

```
from collections import deque

def predict_party_victory(senate):
    """LeetCode 649. Two queues of TURN NUMBERS. The smaller number acts first, bans the other, and
    comes back n turns later."""
    n = len(senate)
    radiant, dire = deque(), deque()
    for i, s in enumerate(senate):
        (radiant if s == 'R' else dire).append(i)
    while radiant and dire:
        r, d = radiant.popleft(), dire.popleft()
        if r < d:
            radiant.append(r + n)           # R acts first, bans this D, and votes again
            next round
        else:
            dire.append(d + n)
    return "Radiant" if radiant else "Dire"
```

Complexity. Each iteration removes one senator, so at most n iterations: **$O(n)$ time, $O(n)$ space**. It seems complicated as a string simulation and becomes *two queues + cyclic positions*.

2.6 Reveal Cards In Increasing Order (LeetCode 950)

Reorder a deck so that this process reveals the cards in **increasing** order: reveal the top card, move the next top card to the bottom, repeat until all are revealed. `[17,13,11,2,3,5,7] → [2,13,3,11,5,17,7]`.

A **reverse queue simulation**. Don't move the cards — simulate the process on positions.

- Put the positions `0..n-1` in a queue.
- Run the process: *reveal* the front position (record it), then move the next position to the back.
- That yields the order in which positions get revealed. Give the **smallest** card to the first revealed position, the next smallest to the second, and so on.

step	reveal (top)	moved to the bottom	positions still hidden
1	position 0	position 1	[2, 3, 4, 5, 6, 1]
2	position 2	position 3	[4, 5, 6, 1, 3]
3	position 4	position 5	[6, 1, 3, 5]
4	position 6	position 1	[3, 5, 1]
5	position 3	position 5	[1, 5]
6	position 1	position 5	[5]
7	position 5	-	[]

Hand the sorted cards to the positions in reveal order:

reveal #	position	card given
1	0	2
2	2	3
3	4	5
4	6	7
5	3	11
6	1	13
7	5	17

Result: **[2, 13, 3, 11, 5, 17, 7]**

```

from collections import deque

def deck_revealed_increasing(deck):
    """LeetCode 950. Run the reveal process on POSITIONS, then hand the sorted cards to the
    positions in reveal order."""
    n = len(deck)
    order = deque(range(n))                                # positions still hidden, top of the deck
    first
    reveal_order = []
    while order:
        reveal_order.append(order.popleft())                # reveal the top card
        if order:
            order.append(order.popleft())                    # move the next card to the bottom
    result = [0] * n
    for card, pos in zip(sorted(deck), reveal_order):
        result[pos] = card
    return result

```

Complexity. $O(n \log n)$ for the sort; the simulation is $O(n)$.

2.7 Design Hit Counter (LeetCode 362, premium)

Count hits in the past 5 minutes (300 seconds). `hit(timestamp)` records a hit; `getHits(timestamp)` returns the hits in `[timestamp - 300, timestamp]`. Calls arrive in non-decreasing time order.

It is Number of Recent Calls with a different window: the same deque, the same "pop the too-old front" loop. (If hits can be extremely dense, store `(second, count)` pairs instead of one entry per hit.)

```
from collections import deque

class HitCounter:
    """LeetCode 362. Hits in the last 300 seconds (timestamps non-decreasing)."""

    def __init__(self):
        self.q = deque()

    def hit(self, timestamp):
        self.q.append(timestamp)

    def getHits(self, timestamp):
        while self.q and self.q[0] <= timestamp - 300:
            self.q.popleft()
        return len(self.q)
```

2.8 Implement a Stack Using Queues (LeetCode 225)

Build a LIFO stack with only queue operations. Using **one** queue: after each `push`, rotate the older elements behind the new one, so the newest is always at the front.

```

from collections import deque

class MyStack:
    """LeetCode 225 - a stack from ONE queue: after each push, rotate the older elements behind the
    new one."""

    def __init__(self):
        self.q = deque()

    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self):
        return self.q.popleft()

    def top(self):
        return self.q[0]

    def empty(self):
        return not self.q

```

`push` is $O(n)$, `pop` / `top` are $O(1)$. (The mirror of §1.6's two-stack queue; a queue can be turned into a stack, but the cost has to be paid *somewhere*.)

2.9 Pattern 1 summary

Problem	What is the queue?	Stop condition	Trick
Time Needed to Buy Tickets	person indices	target person finishes	$O(n)$ formula
Number of Recent Calls	timestamps in the window	never (a stream)	pop the too-old front
Moving Average	last <code>size</code> values	never	maintain a running sum
Students Unable to Eat	students waiting	one full lap without a taker	only <i>counts</i> matter
Dota2 Senate	turn numbers per party	one party's queue is empty	<code>+n</code> for the next round
Reveal Cards	positions	queue is empty	simulate on positions, then assign sorted cards

Part 3 · Pattern 2 — BFS (Breadth-First Search)

Recognition. Process a tree or graph level by level, or find the minimum number of steps / moves / edges when every move costs the same.

3.1 The idea

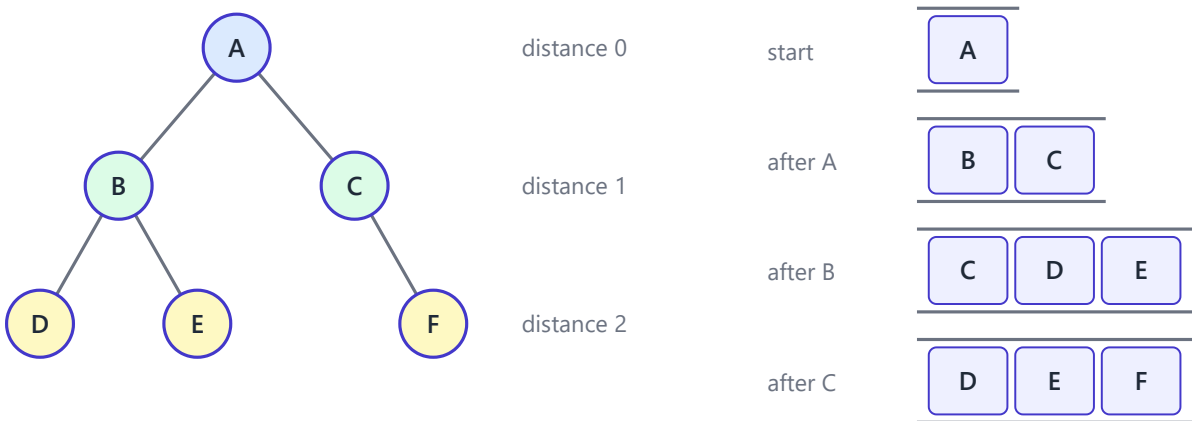
Forget trees and graphs for a moment. You stand at **A** and can move to **B** or **C**; from **B** to **D** or **E**; from **C** to **F**. If you want everything **one step away first**, then everything **two steps away**, in what order must you process the nodes?

```
Distance 0: A
Distance 1: B C
Distance 2: D E F
```

Why does a **queue** produce exactly this order?

Put the nodes you discover at distance 1 into the queue. Process them in FIFO order; while processing them you discover distance-2 nodes and append them to the **back**.

BFS from A: the queue keeps the distance-1 nodes ahead of the distance-2 nodes



process (dequeue)	distance	discovered → enqueued	queue after
A	0	B, C	[B, C]
B	1	D, E	[C, D, E]
C	1	F	[D, E, F]
D	2	-	[E, F]
E	2	-	[F]
F	2	-	[]

Notice that **D E F** were added *behind* **B C** — **FIFO stops a newly discovered distance-2 node from jumping ahead of a distance-1 node that is already waiting.**

The key invariant.

Nodes closer to the source are processed before nodes farther from the source (the queue holds nodes in **non-decreasing distance**).

3.2 The BFS template

```
from collections import deque

q = deque([start])
visited = {start}          # mark when you ENQUEUE (see §6.2)
while q:
    node = q.popleft()
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            q.append(neighbor)
```

```
from collections import deque

def bfs_order(graph, start):
    """The generic BFS template: mark visited when you ENQUEUE."""
    q = deque([start])
    visited = {start}
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                q.append(nb)
    return order
```

`visited` is essential on a graph (without it a cycle loops forever). On a **tree** it is unnecessary because there is exactly one path to each node.

Complexity. Every node is enqueued once and every edge examined once: **$O(V + E)$** time, $O(V)$ space.

3.3 Level-order traversal (LeetCode 102)

Return the values of a binary tree level by level. `[3,9,20,null,null,15,7] → [[3],[9,20],[15,7]]`.

Derive it.

1. What does one queue item represent? A node waiting to be processed, with earlier levels ahead of later levels.
2. What do we add when processing a node? Its children.

3. How do we know which nodes belong to the same level? **They are at the same distance from the root** — but the queue does not label them. The trick:

At the **beginning of each level**, the queue contains *exactly* the nodes of that level. Record `level_size = len(queue)` and process **exactly that many** nodes. The children they add are behind the boundary and wait for the next iteration.

`level_size = len(queue)` freezes the boundary between one level and the next



level	queue at the start	level_size	collected	queue after the level
0	[3]	1	[3]	[9, 20]
1	[9, 20]	2	[9, 20]	[15, 7]
2	[15, 7]	2	[15, 7]	[]

```
put the starting node(s) in the queue
while queue:
    level_size = len(queue)      # <- freeze the boundary
    repeat level_size times:
        take a node from the front
        process it
        add its unvisited children / neighbours to the back
```

```

from collections import deque

def level_order(root):
    """LeetCode 102. `level_size` freezes the boundary between the current level and the next."""
    if not root:
        return []
    result, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            # exactly the nodes that were queued when the
            level started
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        result.append(level)
    return result

```

A helper used by the tree problems (LeetCode-style level-order input, `None` for a missing child):

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def build_tree(vals):
    """LeetCode-style level-order list with None for missing children."""
    if not vals or vals[0] is None:
        return None
    root = TreeNode(vals[0])
    q = deque([root])
    i = 1
    while q and i < len(vals):
        node = q.popleft()
        if i < len(vals) and vals[i] is not None:
            node.left = TreeNode(vals[i])
            q.append(node.left)
        i += 1
        if i < len(vals) and vals[i] is not None:
            node.right = TreeNode(vals[i])
            q.append(node.right)
        i += 1
    return root

```

Two BFS ideas, one queue.

- Ordinary BFS: explore states in non-decreasing distance. **FIFO alone guarantees this.**

- *Level-order BFS*: additionally group the nodes **by** distance. That is what `level_size` is for.

Why don't we process D right after B adds it? Because we stored `level_size = 2` before the level began; after processing exactly two nodes the loop ends, and D, E, F are next level's work.

3.4 Variants that only change what you do per level

Problem	Per level, do this
Right Side View (199)	keep the last node of each level
Zigzag Level Order (103)	reverse every second level
Average of Levels (637)	sum the level and divide by <code>level_size</code>
Minimum Depth (111)	return the depth of the first leaf encountered

```
from collections import deque

def right_side_view(root):
    """LeetCode 199: the LAST node of every level is what you see from the right."""
    if not root:
        return []
    view, q = [], deque([root])
    while q:
        size = len(q)
        for i in range(size):
            node = q.popleft()
            if i == size - 1:
                view.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
    return view
```

```

def zigzag_level_order(root):
    """LeetCode 103: level order, reversing every second level."""
    if not root:
        return []
    result, q, left_to_right = [], deque([root]), True
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        result.append(level if left_to_right else level[::-1])
        left_to_right = not left_to_right
    return result

```

```

def average_of_levels(root):
    """LeetCode 637."""
    result, q = [], deque([root] if root else [])
    while q:
        n = len(q)
        total = 0
        for _ in range(n):
            node = q.popleft()
            total += node.val
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        result.append(total / n)
    return result

```

Minimum Depth is a nice illustration of *why BFS beats DFS for "nearest" questions*: BFS stops the moment it sees the first leaf, without exploring deeper levels of other branches.

```
def min_depth(root):
    """LeetCode 111: BFS stops at the FIRST leaf it meets, which is why BFS beats DFS here."""
    if not root:
        return 0
    q, depth = deque([root]), 1
    while q:
        for _ in range(len(q)):
            node = q.popleft()
            if not node.left and not node.right:
                return depth
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        depth += 1
```

3.5 Shortest path in an unweighted graph

Suppose we want the fewest edges from **A** to **E** in **A – B – D** and **A – C – E**.

1. Distance 1 from **A**? **B** and **C**. 2. Distance 2? **D** and **E**. 3. When **BFS** first reaches **E**, why is that distance the shortest?

BFS explores nodes in increasing order of distance. When it first reaches **E** at distance 2 there cannot be a path of distance 1, because **BFS** would have processed all distance-1 nodes first.

dequeue	dist	newly discovered (node = distance)	queue after
A	0	B=1, C=1	[B, C]
B	1	D=2	[C, D]
C	1	E=2	[D, E]
D	2	-	[E]
E	2	-	[]

The dictionary **dist** doubles as the visited set:

```

from collections import deque

def bfs_distances(graph, start):
    """Shortest number of edges from `start` to every reachable node (unweighted graph)."""
    dist = {start: 0}
    q = deque([start])
    while q:
        node = q.popleft()
        for nb in graph[node]:
            if nb not in dist:
                # `dist` doubles as the visited set
                dist[nb] = dist[node] + 1
                q.append(nb)
    return dist

```

Recover the path itself. Remember *who discovered each node* (`parent`), then walk back from the goal:

```

def bfs_path(graph, start, goal):
    """The path itself, not just its length: remember who discovered each node (`parent`) and walk back."""
    parent = {start: None}
    q = deque([start])
    while q:
        node = q.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nb in graph[node]:
            if nb not in parent:
                parent[nb] = node
                q.append(nb)
    return None

```

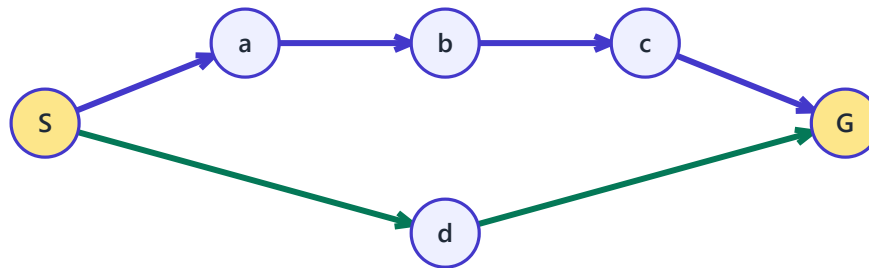
Recognition rule. "Find the minimum number of steps / moves / edges to reach *X*" and every move costs the same → **BFS**.

3.6 Why not DFS for the shortest path?

DFS can tell you whether `E` is **reachable**, but it is not guaranteed to find the **shortest** route: it can go deep along one path before trying shorter alternatives.

First arrival = shortest path (BFS) - DFS may reach the goal the long way round

DFS that explores 'a' first finds S-a-b-c-G (4 edges)



BFS reaches G at distance 2: S-d-G

Question	Use
Can I reach it? Is it connected?	DFS or BFS
Minimum number of moves / edges?	BFS (first arrival = shortest)
Many starting points spreading at once?	multi-source BFS (Part 4)
Extra condition changes future movement?	BFS + state (Part 5)

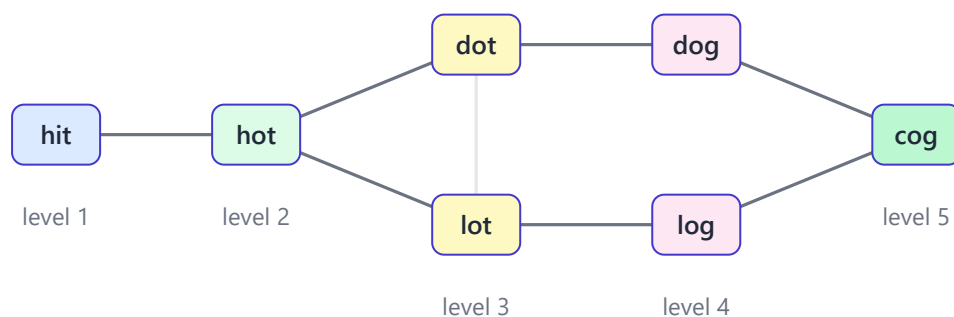
3.7 Word Ladder (LeetCode 127)

Given `beginWord`, `endWord` and a word list, return the number of words in the shortest transformation sequence (change one letter at a time, every intermediate word must be in the list), or `0`. "hit" → "cog" with ["hot", "dot", "dog", "lot", "log", "cog"] → 5.

The graph is not given — it is implicit.

- What is a node? A word. When are two nodes connected? When they differ in exactly one letter. What does a move cost? 1 → **BFS**.
- We don't build the graph: for each word we generate every one-letter change (`L` positions × 25 letters) and check membership in a set.

Word Ladder: every word is a node, one-letter changes are edges



hit → hot → dot → dog → cog : 5 words, so the answer is 5

distance (words counted)	words first reached at this distance
1	hit
2	hot

distance (words counted)	words first reached at this distance
3	dot, lot
4	dog, log
5	cog

cog appears at distance 5.

```
from collections import deque

def ladder_length(beginWord, endWord, wordList):
    """LeetCode 127. Node = word, edge = words that differ in one letter; the graph is generated on the fly."""
    words = set(wordList)
    if endWord not in words:
        return 0
    q = deque([(beginWord, 1)])
    visited = {beginWord}
    while q:
        word, dist = q.popleft()
        if word == endWord:
            return dist
        for i in range(len(word)):
            for ch in 'abcdefghijklmnopqrstuvwxyz':
                if ch == word[i]:
                    continue
                nxt = word[:i] + ch + word[i + 1:]
                if nxt in words and nxt not in visited:
                    visited.add(nxt)
                    q.append((nxt, dist + 1))
    return 0
```

Complexity. Each of the N words is dequeued once; generating its $26 \cdot L$ candidates costs $O(L)$ each (building a new string) $\rightarrow O(26 \cdot N \cdot L^2)$, space $O(N \cdot L)$.

Correction. The session states $O(N \times L \times 26)$. Building `word[:i] + ch + word[i+1:]` is itself $O(L)$, so the real bound has another factor of L .

Faster: bidirectional BFS. Search from *both* ends and always expand the **smaller** frontier; they meet in the middle, exploring roughly the square root of the states.

```

def ladder_length_bidirectional(beginWord, endWord, wordList):
    """LeetCode 127, faster: grow the SMALLER frontier from both ends until they touch."""
    words = set(wordList)
    if endWord not in words:
        return 0
    front, back = {beginWord}, {endWord}
    dist = 1
    words.discard(beginWord)
    while front and back:
        if len(front) > len(back):
            front, back = back, front          # always expand the smaller side
        nxt_front = set()
        for word in front:
            for i in range(len(word)):
                for ch in 'abcdefghijklmnopqrstuvwxyz':
                    cand = word[:i] + ch + word[i + 1:]
                    if cand in back:
                        return dist + 1          # the two searches met
                    if cand in words:
                        words.discard(cand)
                        nxt_front.add(cand)
        front = nxt_front
        dist += 1
    return 0

```

3.8 Open the Lock (LeetCode 752)

A lock has four wheels 0–9 that wrap around. Start at "0000" ; each move turns one wheel by +1 or -1 . Some combinations are **dead ends** (you get stuck there). Return the minimum number of moves to reach target , or -1 .

The state itself is the lock combination. Nodes: the 10^4 combinations. Edges: 8 neighbours (4 wheels $\times$ 2 directions). Deadends are simply **never entered** (treat them as visited from the start).

```

from collections import deque

def open_lock(deadends, target):
    """LeetCode 752. The STATE is the four-digit combination; each move turns one wheel by +1 or
    -1."""
    dead = set(deadends)
    if "0000" in dead:
        return -1
    q = deque([("0000", 0)])
    seen = {"0000"}
    while q:
        code, moves = q.popleft()
        if code == target:
            return moves
        for i in range(4):
            d = int(code[i])
            for nd in ((d + 1) % 10, (d - 1) % 10):
                nxt = code[:i] + str(nd) + code[i + 1:]
                if nxt not in seen and nxt not in dead:
                    seen.add(nxt)
                    q.append((nxt, moves + 1))
    return -1

```

Complexity. $O(10^4 \cdot 8)$ — the state space is tiny even though the "graph" was never given.

3.9 Minimum Genetic Mutation (LeetCode 433)

Word Ladder over the alphabet A C G T with 8-character genes: same BFS, bank is the dictionary.

```

def min_mutation(startGene, endGene, bank):
    """LeetCode 433 - Word Ladder over the alphabet ACGT."""
    bank = set(bank)
    if endGene not in bank:
        return -1
    q = deque([(startGene, 0)])
    seen = {startGene}
    while q:
        gene, steps = q.popleft()
        if gene == endGene:
            return steps
        for i in range(len(gene)):
            for ch in "ACGT":
                nxt = gene[:i] + ch + gene[i + 1:]
                if nxt in bank and nxt not in seen:
                    seen.add(nxt)
                    q.append((nxt, steps + 1))
    return -1

```

3.10 Perfect Squares (LeetCode 279)

The least number of perfect squares that sum to n : $12 \rightarrow 3$ ($4 + 4 + 4$).

Another *implicit graph*: node = **the amount still to be paid**, edge = subtract a perfect square. The shortest path from n to 0 is the answer.

```
def num_squares(n):
    """LeetCode 279. Node = remaining amount, edge = subtract a perfect square; shortest path from n
    to 0."""
    squares = [i * i for i in range(1, int(math.isqrt(n)) + 1)]
    q = deque([(n, 0)])
    seen = {n}
    while q:
        rem, steps = q.popleft()
        for s in squares:
            nxt = rem - s
            if nxt == 0:
                return steps + 1
            if nxt > 0 and nxt not in seen:
                seen.add(nxt)
                q.append((nxt, steps + 1))
    return 0
```

(There is a well-known $O(\sqrt{n})$ number-theory solution — Lagrange's four-square theorem — but the BFS is the one worth deriving.)

3.11 Snakes and Ladders (LeetCode 909)

Board $n \times n$ numbered in boustrophedon (zig-zag) order; each roll takes you 1–6 squares ahead and you must follow a snake or ladder if you land on one. Fewest rolls to reach square n^2 .

Node = square number $1..n^2$. *Edges* = the 6 dice outcomes, followed by any forced jump. The one fiddly part is converting a square number to `(row, col)`:

```

from collections import deque

def snakes_and_ladders(board):
    """LeetCode 909. Node = square number 1..n*n, edges = the 6 dice outcomes (after following a
    snake or ladder)."""
    n = len(board)

    def cell(square):
        # square number -> (row, col) on the
        # boustrophedon board
        r, c = divmod(square - 1, n)
        row = n - 1 - r
        col = c if r % 2 == 0 else n - 1 - c
        return row, col

    q = deque([(1, 0)])
    seen = {1}
    while q:
        sq, rolls = q.popleft()
        for step in range(1, 7):
            nxt = sq + step
            if nxt > n * n:
                continue
            r, c = cell(nxt)
            if board[r][c] != -1:
                nxt = board[r][c]
                # snake or ladder: forced jump
            if nxt == n * n:
                return rolls + 1
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, rolls + 1))

    return -1

```

3.12 Pattern 2 summary

Problem	Node	Edge	Note
Level Order	tree node	child	level_size boundary
Shortest path (unweighted)	graph node	edge	first arrival = shortest
Word Ladder	word	one-letter change	implicit graph
Open the Lock	4-digit code	turn a wheel	deadends = pre-visited
Perfect Squares	remaining amount	subtract a square	implicit graph
Snakes and Ladders	square number	dice roll (+ jump)	zig-zag numbering

Part 4 · Pattern 3 — Multi-Source BFS

Recognition. Several nodes are sources at the same time, and we want the minimum distance from any source — nearest rotten orange, nearest hospital, nearest gate, spread of a fire.

4.1 The idea

A B Instead of: "start from one source and expand"
 \ / we do: "start from ALL sources at the same time and expand
together"
 ...

How do you initialise the queue? With **both** A and B in it, both at distance 0. The queue handles the rest by itself: it expands outward from every source simultaneously, level by level.

Suppose A reaches node x in 3 steps but B would need 5. Which distance should x get? **3**. By the time the search from B reaches distance 5, x has already been discovered at distance 3.

Invariant. Multi-source BFS treats every initial source as distance 0 and expands outward together, so **each node is reached at its minimum distance from any source**.

Multi-source BFS: every source starts at distance 0, the waves grow together

0	1	2	1	0
1	2	3	2	1
2	3	4	3	2
3	4	3	2	1
4	3	2	1	0

sources $S = 0$ (all in the queue at the start)

level 1 = cells one step from any source

a cell keeps the FIRST distance that reaches it,
which is its minimum over all sources

0 1 2 3 4

```

from collections import deque
q = deque()
for source in sources:
    q.append(source)
    visited[source] = True
    distance[source] = 0
while q:
    node = q.popleft()
    for neighbor in graph[node]:
        if not visited[neighbor]:
            visited[neighbor] = True
            distance[neighbor] = distance[node] + 1
            q.append(neighbor)

```

The reversal to remember. A tempting solution runs a separate BFS *from every cell* until it finds a source — enormous repeated work. **Reverse the perspective:** start from the sources and let the waves find the cells.

4.2 Rotting Oranges (LeetCode 994)

Grid: 0 empty, 1 fresh orange, 2 rotten orange. Every minute each fresh orange next to a rotten one rots. Return the minutes until no fresh orange is left, or -1.

Sources = every rotten orange. One BFS level = one minute. The answer is the number of levels needed to reach every fresh orange; if a fresh one is left over, it was unreachable → -1.

minute	grid (🍌 rotten, 🍊 fresh, . empty)	note
0		the rotten orange is the only source
1		4 fresh left
2		2 fresh left
3		1 fresh left
4		0 fresh left

```

from collections import deque

def oranges_rotting(grid):
    """LeetCode 994. Every rotten orange is a source at minute 0; the answer is the number of BFS
    levels."""
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in DIRS4:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
            minutes += 1
    return minutes if fresh == 0 else -1

```

Two subtleties. (1) Turning the orange into `2` is the visited mark. (2) The loop stops as soon as `fresh == 0`, otherwise we would count one extra, empty minute.

4.3 01 Matrix (LeetCode 542)

For every cell, the distance to the nearest `0` (4-directional).

Don't run BFS from every `1`. **All the `0`s form the initial queue**; each `1` gets the level at which the wave first reaches it.

BFS level	cells at this distance from the nearest 0
level 0	(0,0), (0,1), (0,2), (1,0), (1,2)
level 1	(1,1), (2,0), (2,2)
level 2	(2,1)

Result: `[[0, 0, 0], [0, 1, 0], [1, 2, 1]]`

```

from collections import deque

def update_matrix(mat):
    """LeetCode 542. Sources = all the 0s. Every 1 gets the level at which the wave first reaches it."""
    rows, cols = len(mat), len(mat[0])
    dist = [[-1] * cols for _ in range(rows)]
    q = deque()
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                dist[r][c] = 0
                q.append((r, c))
    while q:
        r, c = q.popleft()
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist

```

Complexity. $O(\text{rows} \cdot \text{cols})$ time and space — every cell is enqueued once.

4.4 Walls and Gates (LeetCode 286, premium)

Rooms: -1 wall, 0 gate, $2^{31} - 1$ (INF) empty room. Fill every empty room with the distance to its nearest gate (leave it INF if unreachable).

The mirror of 01 Matrix: the sources are the **gates**, the walls block, and INF doubles as the "unvisited" marker.

```

from collections import deque

def walls_and_gates(rooms):
    """LeetCode 286. Sources = gates (0); INF = 2**31 - 1 is an empty room, -1 is a wall. Fills `rooms` in place."""
    INF = 2 ** 31 - 1
    rows, cols = len(rooms), len(rooms[0]) if rooms else 0
    q = deque((r, c) for r in range(rows) for c in range(cols) if rooms[r][c] == 0)
    while q:
        r, c = q.popleft()
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == INF:
                rooms[nr][nc] = rooms[r][c] + 1
                q.append((nr, nc))
    return rooms

```

4.5 As Far from Land as Possible (LeetCode 1162)

Grid `1` = land, `0` = water. Find the water cell whose distance to the nearest land is **maximum**; return that distance (or `-1` if there is no water or no land).

Again: *all land cells* → *initial queue*, then BFS outward. **The last BFS level is the answer** — the final wave reaches the water cell that is farthest from all land.

```
from collections import deque

def max_distance(grid):
    """LeetCode 1162. Sources = all land; the LAST BFS level is the water cell farthest from any land."""
    n = len(grid)
    q = deque((r, c) for r in range(n) for c in range(n) if grid[r][c] == 1)
    if len(q) in (0, n * n):
        return -1
    dist = -1
    while q:
        dist += 1
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in DIRS4:
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                    grid[nr][nc] = 1
                    q.append((nr, nc))
    return dist
```

4.6 Pattern 3 summary

Problem	Sources	What a level means	Answer
Rotting Oranges	rotten oranges	one minute	number of levels (or <code>-1</code>)
01 Matrix	all <code>0</code> cells	distance	the level of each cell
Walls and Gates	gates	distance	the level of each room
As Far from Land	land cells	distance	the last level

Part 5 · Pattern 4 — BFS + State

Recognition. *Position alone does not describe the situation — some extra information changes what you can do next.*

```
(row, col, walls_broken)    (node, keys_collected)
(node, fuel_remaining)      (position, powerups_used)
```

5.1 The idea

You move through a grid but may **break one wall**. A normal BFS state is `(row, col)`. **Is that enough?** No — reaching the same cell **before** and **after** spending the wall break are different situations:

```
X, 0    ← reached X without breaking a wall  (you can still break one later)
X, 1    ← reached X after breaking a wall    (you cannot)
```

So the state must be `(row, col, wall_break_used)`.

If the future possibilities depend on some extra information, that information becomes part of the BFS state.

Ask: what information must travel with me because it changes my future choices?

```
from collections import deque
q = deque([(start, initial_state)])
visited = {(start, initial_state)}
while q:
    position, state = q.popleft()
    for next_position, next_state in get_next_states(position, state):
        if (next_position, next_state) not in visited:
            visited.add((next_position, next_state))
            q.append((next_position, next_state))
```

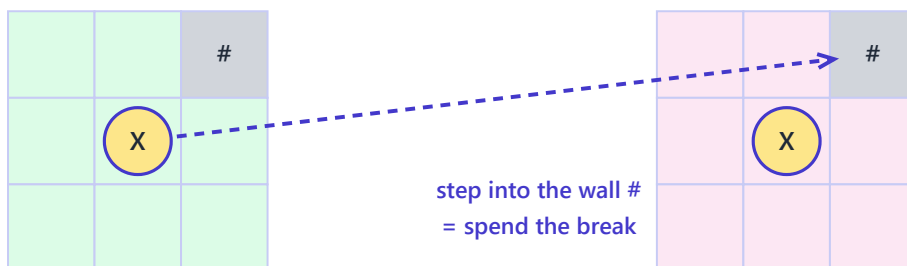
5.2 What does *visited* mean here?

Would visiting `(5, 5, 0)` mean you have also visited `(5, 5, 1)`? **No** — their futures differ.

BFS + state: the same cell in two different states is two different nodes

layer used = 0 (break still available)

layer used = 1 (break already spent)



(X,0) and (X,1) are different nodes: from (X,0) you can still break a wall later, from (X,1) you cannot

The entire BFS state determines what "visited" means. Instead of `visited[row][col]` we need `visited[row][col][wall_break_used]`.

You can picture the state graph as **layers**: layer 0 is the grid before spending the break, layer 1 is the grid after. Ordinary moves stay inside a layer; stepping into a wall jumps from layer 0 to layer 1. BFS then finds the shortest path through this bigger, two-layer graph.

5.3 Break at most one wall

Grid: 0 open, 1 wall. Move up/down/left/right from the top-left to the bottom-right; you may pass through **at most one** wall. Return the fewest moves, or `-1`.

Transitions from `(r, c, used)` to a neighbour `(nr, nc)`:

Neighbour	Condition	New state
open cell	—	<code>(nr, nc, used)</code>
wall	<code>used == 0</code> (break available)	<code>(nr, nc, 1)</code>
wall	<code>used == 1</code>	not allowed

dequeued state (row, col, used)	distance	queue after (states)
(0,0,0)	0	[]
(1,0,1)	1	[(0,1,1)]
(0,1,1)	1	[(0,0,1), (2,0,1)]
(0,0,1)	2	[(2,0,1), (0,2,1)]
(2,0,1)	2	[(0,2,1)]
(0,2,1)	2	[(2,1,1)]
(2,1,1)	3	[(1,2,1)]
(1,2,1)	3	[(2,2,1)]
(2,2,1)	4	[]

The start `(0,0)` is walled in on both sides, so the only way out is to spend the break at once; the search never needs `used = 0` again.

```
from collections import deque

def shortest_path_one_break(grid):
    """The session's wall-break problem, corrected. 0 = open, 1 = wall, you may break AT MOST ONE
    wall.
    State = (row, col, wall_break_used); visited is keyed by the whole state."""
    rows, cols = len(grid), len(grid[0])
    q = deque([(0, 0, 0, 0)]) # row, col, used, distance
    visited = [[[False] * 2 for _ in range(cols)] for _ in range(rows)]
    visited[0][0][0] = True
    while q:
        r, c, used, dist = q.popleft()
        if r == rows - 1 and c == cols - 1:
            return dist
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == 0:
                nu = used # open cell: nothing consumed
            elif used == 0:
                nu = 1 # wall and the break is still available:
            consume it
            else:
                continue # wall and no break left
            if not visited[nr][nc][nu]: # the check the original code skipped for the
            wall branch
                visited[nr][nc][nu] = True
                q.append((nr, nc, nu, dist + 1))
    return -1
```

Correction. In the session's code, the wall branch was `python elif grid[nr][nc] == 1 and used == 0: visited[nr][nc][1] = True # set, but never CHECKED queue.append((nr, nc, 1, distance + 1))` It never tested `visited[nr][nc][1]` before enqueueing, so a state could enter the queue several times (still correct, but wasteful — and it violates the invariant "if a state is in the queue it is already marked visited, exactly once"). The version above tests `visited` for **both** branches. The session also called `1` "the starting cell" in one example; `1` is a **wall** and the start is a separate open cell.

5.4 Break at most K walls — Shortest Path in a Grid with Obstacles Elimination (LeetCode 1293)

`0` empty, `1` obstacle; you may eliminate **at most** `k` obstacles. Fewest steps from `(0,0)` to `(m-1, n-1)`, or `-1`.

The boolean becomes a **count**: `(row, col, obstacles_eliminated)` with $0 \leq \text{eliminated} \leq k$. These are $k + 1$ different states per cell:

<code>(2, 4, 0)</code> haven't broken any	<code>(2, 4, 2)</code> broken 2
<code>(2, 4, 1)</code> broken 1	<code>(2, 4, 3)</code> broken 3

Transitions: open cell $\rightarrow$ same count; obstacle and $b < k \rightarrow b + 1$; obstacle and $b == k \rightarrow$ not allowed.

```
from collections import deque

def shortest_path_k(grid, k):
    """LeetCode 1293 - at most k obstacles may be removed; state = (row, col, obstacles_removed)."""
    rows, cols = len(grid), len(grid[0])
    if k >= rows + cols - 2:
        return rows + cols - 2  # the Manhattan path always exists
    q = deque([(0, 0, 0, 0)])
    visited = {(0, 0, 0)}
    while q:
        r, c, used, dist = q.popleft()
        if r == rows - 1 and c == cols - 1:
            return dist
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            nu = used + grid[nr][nc]  # 1 for an obstacle, 0 for an open cell
            if nu > k:
                continue
            state = (nr, nc, nu)
            if state not in visited:
                visited.add(state)
                q.append((nr, nc, nu, dist + 1))
    return -1
```

Complexity. There are $\text{rows} \times \text{cols} \times (k + 1)$ states, each with 4 edges: $O(\text{rows} \cdot \text{cols} \cdot k)$ time and space.

Two optimisations worth knowing. (1) $k \geq \text{rows} + \text{cols} - 2$ means the plain Manhattan path always exists, so the answer is $\text{rows} + \text{cols} - 2$ immediately (the first line of the code). (2) A state (r, c, b) is *dominated* by (r, c, b') with $b' < b$ reached at the same or a shorter distance — BFS order already makes the first arrival at each count optimal.

The general lesson. A limited resource that affects future decisions becomes part of the BFS state — keys, power-ups, fuel, discounts, turns remaining.

5.5 Shortest Path to Get All Keys (LeetCode 864)

Grid symbols: @ start, . empty, # wall, a–f keys, A–F locks. You may enter lock A only after picking up key a. Fewest moves to collect **all** keys.

Position is now nowhere near enough. The state must say **which keys you hold**. With at most 6 keys, encode the set as a **bitmask**: bit `i` is set when you own key `chr(ord('a') + i)`.

```
state = (row, col, keys_mask)          keys_mask ∈ [0, 2^6) → at most 64 layers
goal = keys_mask == full_mask
```

- Stepping onto a key **sets** its bit (`mask | 1 << i`).
- Stepping onto a lock is allowed only if the corresponding bit is set.

This problem shows that *the state space itself is the challenge*: $30 \times 30 \text{ cells} \times 64 \text{ masks} \approx 57\,600 \text{ states}$.

```
from collections import deque

def shortest_path_all_keys(grid):
    """LeetCode 864. State = (row, col, keys_mask): bit i set means key 'a'+i is in your pocket."""
    rows, cols = len(grid), len(grid[0])
    start, full = None, 0 # 'full' = the mask with every key on the map
    set
    for r in range(rows):
        for c in range(cols):
            ch = grid[r][c]
            if ch == '@':
                start = (r, c)
            elif ch.islower():
                full |= 1 << (ord(ch) - ord('a'))
    q = deque([(start[0], start[1], 0, 0)])
    seen = {(start[0], start[1], 0)}
    while q:
        r, c, keys, dist = q.popleft()
        if keys == full:
            return dist
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            ch = grid[nr][nc]
            if ch == '#':
                continue
            nk = keys
            if ch.islower():
                nk |= 1 << (ord(ch) - ord('a')) # pick the key up
            elif ch.isupper() and not (keys >> (ord(ch) - ord('A'))) & 1:
                continue # a lock without its key
            if (nr, nc, nk) not in seen:
                seen.add((nr, nc, nk))
                q.append((nr, nc, nk, dist + 1))
    return -1
```

5.6 Minimum Knight Moves (LeetCode 1197, premium)

A knight starts at $(0, 0)$ on an infinite board. Fewest moves to (x, y) .

The board is infinite but the answer is symmetric in the signs of x and y , so we can search only the region $x \geq -2$, $y \geq -2$ (a margin lets the knight step "backwards" near the axes).

```
from collections import deque

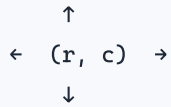
def min_knight_moves(x, y):
    """LeetCode 1197. The board is infinite, but by symmetry we may work in the first quadrant (with
    a small margin)."""
    x, y = abs(x), abs(y)
    moves = [(1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1), (-2, 1), (-1, 2)]
    q = deque([(0, 0, 0)])
    seen = {(0, 0)}
    while q:
        cx, cy, d = q.popleft()
        if (cx, cy) == (x, y):
            return d
        for dx, dy in moves:
            nx, ny = cx + dx, cy + dy
            if (nx, ny) not in seen and nx >= -2 and ny >= -2:  # the margin lets the knight step
                # "backwards" near the axes
                seen.add((nx, ny))
                q.append((nx, ny, d + 1))
```

5.7 Pattern 4 summary

Problem	State	Visited key	Complication
Break one wall	(r, c, used)	$\text{visited}[r][c][\text{used}]$	check <code>visited</code> on the wall branch too
Obstacles Elimination	$(r, c, \text{eliminated})$	$(r, c, \text{eliminated})$	$k + 1$ layers; early exit when k is large
All Keys	$(r, c, \text{keys_mask})$	(r, c, mask)	bitmask; locks need keys
Open the Lock (\$3.8)	the combination	the combination	deadends pre-visited
Knight Moves	(x, y)	(x, y)	infinite board; use symmetry

Part 6 · Pattern 5 — BFS on Grids

Recognition. A grid where each cell is a node and neighbouring cells are connected. Usually the four neighbours:



6.1 The grid as a graph

Grid BFS: each cell is a node; neighbours are the cells you may step to

4 directions

	-1,+0	
+0,-1	r,c	+0,+1
	+1,+0	

8 directions (king moves)

-1,-1	-1,+0	-1,+1
+0,-1	r,c	+0,+1
+1,-1	+1,+0	+1,+1

for every neighbour: 1) inside the grid? 2) allowed to enter? 3) not visited yet (in this state)? -> mark visited, enqueue

For (2, 3) the neighbours are (1,3) up, (3,3) down, (2,2) left, (2,4) right. For **every neighbour** we check, in this order:

1. **Inside the grid?** A neighbour with `row = -1` is discarded.
2. **Allowed to enter?** Not a wall (unless the state allows breaking it).
3. **Not already visited** (in this state)?

Only then do we mark it visited and enqueue it.

```

DIRS4 = [(-1, 0), (1, 0), (0, -1), (0, 1)]          # up, down, left, right
DIRS8 = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]

q = deque([(start_r, start_c)])
visited = {(start_r, start_c)}
while q:
    r, c = q.popleft()
    for dr, dc in DIRS4:
        nr, nc = r + dr, c + dc
        if nr < 0 or nr >= rows or nc < 0 or nc >= cols:
            continue
        if grid[nr][nc] is invalid:
            continue
        if (nr, nc) in visited:
            continue
        visited.add((nr, nc))
        q.append((nr, nc))

```

The shortest-distance version stores the distance in the queue: `q = deque([(start_r, start_c, 0)])`, and pushes `(nr, nc, d + 1)`.

6.2 When do you mark a cell visited?

A. when you **remove** it from the queue, or B. when you **add** it? **B.**

Two different cells can both discover `(2, 3)`. If we wait until it is *removed* to mark it, both `A → (2, 3)` and `B → (2, 3)` enqueue it and the same state is processed twice. So the rule is:

discover → mark visited → enqueue

Invariant. *If a state is in the queue, it is already marked visited.*

The cost of getting this wrong grows quickly — on an **empty** grid:

empty grid	cells	enqueues: mark when ENQUEUED	enqueues: mark when DEQUEUED
4 × 4	16	16	25
8 × 8	64	64	113
16 × 16	256	256	481
32 × 32	1024	1024	1985

```
def bfs_count_enqueues(rows, cols, mark_on_enqueue):
    """Demo for Part 6: how many times is a cell put in the queue on an empty rows x cols grid?"""
    q = deque([(0, 0)])
    seen = {(0, 0)} if mark_on_enqueue else set()
    enqueues = 1
    while q:
        r, c = q.popleft()
        if not mark_on_enqueue:
            if (r, c) in seen:
                continue # a duplicate that was already processed
            seen.add((r, c))
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen:
                if mark_on_enqueue:
                    seen.add((nr, nc)) # claim it NOW so nobody else enqueues
                it
                q.append((nr, nc))
                enqueues += 1
    return enqueues
```

6.3 DFS vs BFS on a grid

```
S . . .
. . # .
. # . E
. . . .
```

Question	Use
Can I reach E from S ? / Is it connected?	DFS or BFS
Minimum number of moves from S to E ?	BFS — it explores by distance
Many starting points spreading at once?	multi-source BFS
An extra condition changes future movement?	BFS + state

6.4 Shortest Path in Binary Matrix (LeetCode 1091)

$n \times n$ grid of 0 / 1. Find the shortest **clear path** (cells all 0) from the top-left to the bottom-right moving in **8 directions**. The length counts **cells**, so the start already has length 1.

Recognise it as *grid cell = node, 8 neighbours = edges, BFS = shortest path*. Just swap **DIRS4** for **DIRS8**.

```

from collections import deque

def shortest_path_binary_matrix(grid):
    """LeetCode 1091. Same BFS, but 8 neighbours; a path counts CELLS, so the start already has
    length 1."""
    n = len(grid)
    if grid[0][0] or grid[n - 1][n - 1]:
        return -1
    q = deque([(0, 0, 1)])
    grid[0][0] = 1 # marks visited (mutates the input)
    while q:
        r, c, d = q.popleft()
        if r == n - 1 and c == n - 1:
            return d
        for dr, dc in DIRS8:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                grid[nr][nc] = 1
                q.append((nr, nc, d + 1))
    return -1

```

6.5 Nearest Exit from Entrance in Maze (LeetCode 1926)

'.' open, '+' wall. Fewest steps from `entrance` to the nearest **exit** — an open cell on the border that is *not* the entrance.

Mark the entrance visited first (so it can never count as an exit), then BFS; the first border cell reached is the answer.

```

from collections import deque

def nearest_exit(maze, entrance):
    """LeetCode 1926. An exit is an open border cell that is not the entrance."""
    rows, cols = len(maze), len(maze[0])
    er, ec = entrance
    q = deque([(er, ec, 0)])
    maze[er][ec] = '+'
    while q:
        r, c, d = q.popleft()
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == '.':
                if nr in (0, rows - 1) or nc in (0, cols - 1):
                    return d + 1
                maze[nr][nc] = '+'
                q.append((nr, nc, d + 1))
    return -1

```

6.6 Number of Islands with BFS (LeetCode 200)

Count the connected groups of '1' s. Every unvisited 1 starts a new island; BFS from it marks the whole island.

```
from collections import deque

def num_islands_bfs(grid):
    """LeetCode 200 with BFS instead of DFS (no recursion limit)."""
    rows, cols = len(grid), len(grid[0])
    seen = [[False] * cols for _ in range(rows)]
    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and not seen[r][c]:
                islands += 1
                seen[r][c] = True
                q = deque([(r, c)])
                while q:
                    cr, cc = q.popleft()
                    for dr, dc in DIRS4:
                        nr, nc = cr + dr, cc + dc
                        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1' and not
seen[nr][nc]:
                            seen[nr][nc] = True
                            q.append((nr, nc))
    return islands
```

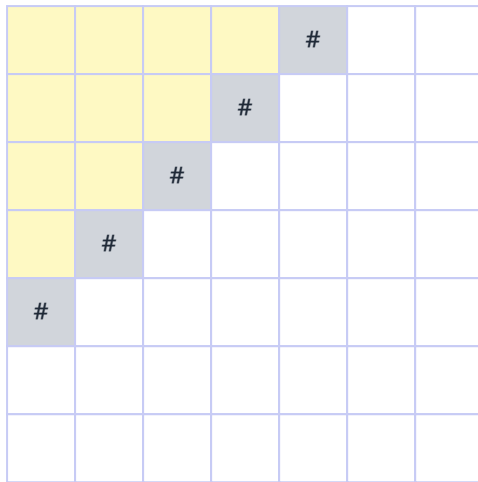
This is *connectivity* (DFS would also do); BFS avoids Python's recursion limit on large grids.

6.7 Escape a Large Maze (LeetCode 1036)

A $10^6 \times 10^6$ grid with at most 200 **blocked** cells. Can you get from `source` to `target`?

A normal BFS could explore 10^{12} cells — impossible. The difficulty is **realising how much of the grid BFS must actually explore**.

Escape a Large Maze: B blocked cells enclose at most $B(B-1)/2$ cells



B = 5 blocked cells on a diagonal in the corner
enclose $4+3+2+1 = 10 = 5 \cdot 4 / 2$ cells (yellow)

so a BFS that has visited MORE than 10 cells
must have escaped every possible enclosure

run it from the source AND from the target

- B blocked cells can enclose at most $B(B-1)/2$ cells (a diagonal wall cutting off a corner).
- So a BFS that has **visited more than that many cells** must have escaped every possible enclosure.
- Either endpoint could be the one that is trapped, so run the bounded BFS **from the source and from the target** — both must escape (or meet).

```
from collections import deque

def is_escape_possible(blocked, source, target, N=10 ** 6):
    """LeetCode 1036. B blocked cells enclose at most  $B(B-1)/2$  cells, so a search that visits MORE
    than that
    has certainly escaped. Both endpoints must escape (or meet)."""
    blocked = {tuple(b) for b in blocked}
    limit = len(blocked) * (len(blocked) - 1) // 2

    def bfs(start, finish):
        start, finish = tuple(start), tuple(finish)
        q = deque([start])
        seen = {start}
        while q and len(seen) <= limit:
            r, c = q.popleft()
            if (r, c) == finish:
                return True
            for dr, dc in DIRS4:
                nr, nc = r + dr, c + dc
                if 0 <= nr < N and 0 <= nc < N and (nr, nc) not in blocked and (nr, nc) not in seen:
                    seen.add((nr, nc))
                    q.append((nr, nc))
        return len(seen) > limit  # escaped the largest possible enclosure

    return bfs(source, target) and bfs(target, source)
```

Complexity. $O(B^2)$ instead of $O(10^{12})$. Here BFS is not the hard part — *deciding when to stop* is.

6.8 Pattern 5 summary

Problem	Twist
Binary Matrix	8 directions; path length counts cells
Nearest Exit	exclude the entrance; mark it visited first
Number of Islands	connectivity — a fresh BFS per component
Escape a Large Maze	bound the search by $B(B-1)/2$, run from both ends
Rotting Oranges / Walls and Gates (Part 4)	multi-source grid BFS
Obstacles Elimination (Part 5)	grid BFS + state

Part 7 · Pattern 6 — Monotonic Queue / Deque

Recognition. *Sliding window and you need the maximum or minimum of every window efficiently.* For a **maximum** keep a **decreasing** deque; for a **minimum**, an **increasing** one. **Store indices, not just values.**

7.1 Deriving it

```
nums = [1, 3, -1, -3, 5, 3, 6, 7]    window size k = 3
window [1, 3, -1] → max 3
window [3, -1, -3] → max 3
```

What information would be useful to keep? Not every element — only the **candidates that could still become the maximum** for the current or a future window.

For `[1, 3, -1]`: once `3` has arrived, can `1` ever be the maximum again while `3` is inside the window? **No** — `3` is larger and stays in the window longer. So `1` is useless forever and can be discarded.

When a new value dominates smaller values behind it, those smaller values are permanently useless and can be removed.

The deque `[5, 3, 2]` and a new value `6`: all of `5, 3, 2` are discarded $\rightarrow [6]$. With `[7, 5, 3]` and `6`: `3 < 6` remove, `5 < 6` remove, `7 > 6` stop $\rightarrow [7, 6]$ (`7` stays: the new `6` cannot dominate it).

7.2 The two removal rules

```
front → EXPIRED : the index is no longer inside the window
back  → DOMINATED: a smaller value that can never beat the newcomer
```

Invariant. *The deque contains only useful, non-expired candidates, in decreasing value order — so the front is always the maximum of the current window.*

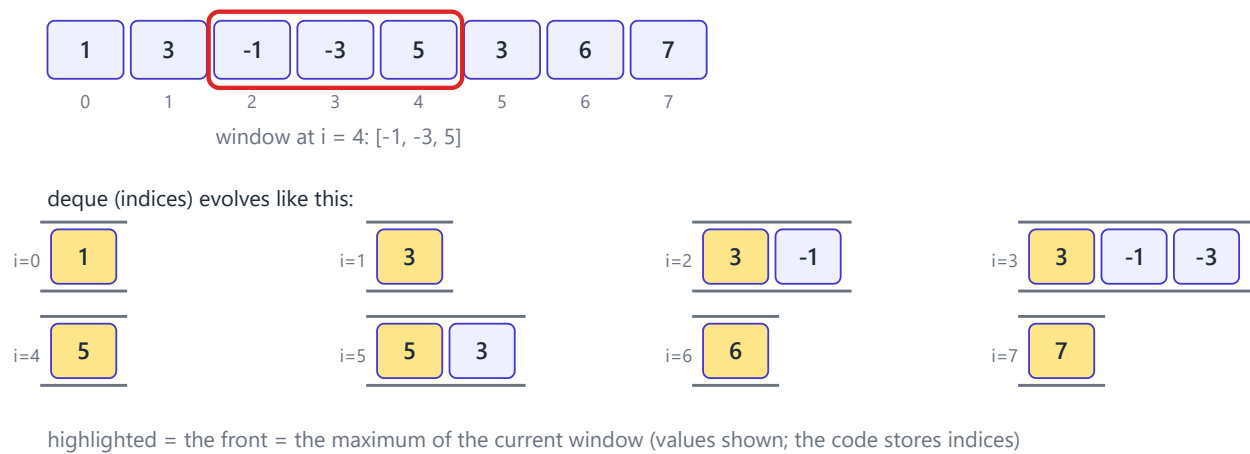
Why indices? The value tells you whether a candidate is *useful*; the **index tells you whether it is still inside the window**. At index 3 with `k = 3` the window is indices `1..3`; a front index of `0` has expired.

Order of the steps for each new element:

1. Remove **expired** indices from the **front** (`front ≤ i - k`).
2. Remove **dominated** elements from the **back** (`nums[back] ≤ x`).
3. Append `i` to the back.
4. Once the window is full (`i ≥ k - 1`), the front is the answer.

(Doing the expiry check before or after the back-removal is equivalent as long as both rules hold.)

Monotonic deque for sliding-window maximum (nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3)



7.3 Sliding Window Maximum (LeetCode 239)

Return the maximum of every window of size k . [1,3,-1,-3,5,3,6,7] , $k = 3 \rightarrow$ [3,3,5,5,6,7] .

i: x	expired (front)	dominated (back)	deque value@index (front $\rightarrow$ back)	window max
0: 1	-	-	[1@0]	-
1: 3	-	1	[3@1]	-
2: -1	-	-	[3@1, -1@2]	3
3: -3	-	-	[3@1, -1@2, -3@3]	3
4: 5	3	-3, -1	[5@4]	5
5: 3	-	-	[5@4, 3@5]	5
6: 6	-	3, 5	[6@6]	6
7: 7	-	6	[7@7]	7

Output: [3, 3, 5, 5, 6, 7]

```

from collections import deque

def max_sliding_window(nums, k):
    """LeetCode 239. The deque holds INDICES whose values are strictly decreasing; the front is the
    window maximum."""
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and dq[0] <= i - k:                # front rule: expired index
            dq.popleft()
        while dq and nums[dq[-1]] <= x:            # back rule: dominated by the newcomer
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

```

Complexity. Each index enters the deque once and leaves once → **O(n) time** (amortized, §1.7), O(k) space. The naive scan is O(n·k).

Ties (<= vs <). With `nums[dq[-1]] <= x` equal values are popped (only the *latest* of equal values is kept). Popping only on `<` keeps equal values too, which is also correct but the deque is then *non-increasing* rather than strictly decreasing.

7.4 Sliding Window Minimum

The mirror image: flip the comparison and keep an **increasing** deque.

```

def min_sliding_window(nums, k):
    """Sliding window MINIMUM: the mirror image - an increasing deque."""
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and dq[0] <= i - k:
            dq.popleft()
        while dq and nums[dq[-1]] >= x:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

```

7.5 Shortest Subarray with Sum at Least K (LeetCode 862)

Shortest **non-empty contiguous** subarray of `nums` (which may contain **negative numbers**) with `sum ≥ k`, or `-1`.

Negatives break the usual two-pointer window (shrinking does not always decrease the sum). Use **prefix sums** — the subarray $[j, i]$ has sum $\text{prefix}[i] - \text{prefix}[j]$; we want the largest $j < i$ with $\text{prefix}[i] - \text{prefix}[j] \geq k$.

- **Front pop (use it, then drop it):** if $\text{prefix}[i] - \text{prefix}[\text{front}] \geq k$ we have a candidate of length $i - \text{front}$. Any later i would give a longer subarray with that front , so front is finished $\rightarrow$ pop it.
- **Back pop (dominated):** if $\text{prefix}[\text{back}] \geq \text{prefix}[i]$, then i is a **later start with an equal or smaller prefix** — it is better in every future window $\rightarrow$ pop back .

The deque therefore holds start indices with **increasing prefix sums**.

Prefix sums of $[2, -1, 2]$ are $[0, 2, 1, 3]$.

i	prefix[i]	front pops (window reaches k)	deque of start indices (prefix)	best length
0	0	-	[0(0)]	-
1	2	-	[0(0), 1(2)]	-
2	1	-	[0(0), 2(1)]	-
3	3	$\text{prefix}[3] - \text{prefix}[0] = 3 \geq 3 \rightarrow$ length 3	[2(1), 3(3)]	3

```
from collections import deque

def shortest_subarray(nums, k):
    """LeetCode 862 (negative numbers allowed). Over PREFIX SUMS: keep an increasing deque of
    candidate start
    indices; pop from the FRONT while the window [front, i] already reaches k, and from the BACK
    while the prefix is not smaller."""
    n = len(nums)
    prefix = [0] * (n + 1)
    for i, x in enumerate(nums):
        prefix[i + 1] = prefix[i] + x
    best = n + 1
    dq = deque()
    for i in range(n + 1):
        while dq and prefix[i] - prefix[dq[0]] >= k: # this start is as good as it will ever get:
            use it, then drop it
            best = min(best, i - dq.popleft())
        while dq and prefix[dq[-1]] >= prefix[i]: # a later start with a smaller prefix
            dominates
            dq.pop()
        dq.append(i)
    return best if best <= n else -1
```

Complexity. $O(n)$ amortized.

7.6 Constrained Subsequence Sum (LeetCode 1425)

The maximum sum of a non-empty subsequence in which any two consecutive chosen elements are at most k indices apart.

This is **DP + sliding window + monotonic deque**:

```
dp[i] = nums[i] + max(0, max(dp[i-k .. i-1]))
```

A naive scan of the previous `k` values costs $O(n \cdot k)$. The inner `max` is a **sliding-window maximum over `dp`** — exactly Pattern 6:

1. *Expired*: `dq[0] < i - k` → pop the front.
2. *Dominated*: `dp[dq[-1]] ≤ dp[i]` → pop the back.

i: nums[i]	best previous dp (front)	dp[i]	deque (front → back)
0: 10	0	10	[dp0=10]
1: 2	10	12	[dp1=12]
2: -10	12	2	[dp1=12, dp2=2]
3: 5	12	17	[dp3=17]
4: 20	17	37	[dp4=37]

Answer = $\max(dp) = 37$ ($10 + 2 + 5 + 20$).

```
from collections import deque

def constrained_subset_sum(nums, k):
    """LeetCode 1425. dp[i] = nums[i] + max(0, max(dp[i-k .. i-1])); the range maximum comes from a
    monotonic deque."""
    dp = [0] * len(nums)
    dq = deque()
    best = float('-inf')
    for i, x in enumerate(nums):
        while dq and dq[0] < i - k:
            dq.popleft()
        dp[i] = x + (max(0, dp[dq[0]]) if dq else 0)
        best = max(best, dp[i])
        while dq and dp[dq[-1]] <= dp[i]:
            dq.pop()
        dq.append(i)
    return best
```

Complexity. Each index enters and leaves the deque once → $O(n)$ time, $O(n)$ space.

7.7 Longest Continuous Subarray with Absolute Diff $\leq$ Limit (LeetCode 1438)

Longest subarray in which $\max - \min \leq \text{limit}$.

Slide a window `[left, right]` and keep **two** deques — the window's maximum (decreasing) and minimum (increasing). While $\max - \min > \text{limit}$, advance `left` and drop expired fronts.

```

from collections import deque

def longest_subarray(nums, limit):
    """LeetCode 1438. Two deques (window max and window min); shrink from the left while max - min > limit."""
    maxd, mind = deque(), deque()
    left = best = 0
    for right, x in enumerate(nums):
        while maxd and nums[maxd[-1]] <= x:
            maxd.pop()
        maxd.append(right)
        while mind and nums[mind[-1]] >= x:
            mind.pop()
        mind.append(right)
        while nums[maxd[0]] - nums[mind[0]] > limit:
            left += 1
            if maxd[0] < left:
                maxd.popleft()
            if mind[0] < left:
                mind.popleft()
        best = max(best, right - left + 1)
    return best

```

7.8 Jump Game VI (LeetCode 1696)

Start at index 0; from i you may jump to any of $i+1 \dots i+k$; the score is the sum of the values you land on. Maximum score at the last index.

$dp[i] = nums[i] + \max(dp[i-k \dots i-1])$ — Constrained Subsequence Sum without the $\max(0, \dots)$ (you *must* land on every jump).

```

from collections import deque

def max_result(nums, k):
    """LeetCode 1696.  $dp[i] = nums[i] + \max(dp[i-k \dots i-1])$  - a monotonic deque over dp."""
    dp = [0] * len(nums)
    dp[0] = nums[0]
    dq = deque([0])
    for i in range(1, len(nums)):
        while dq and dq[0] < i - k:
            dq.popleft()
        dp[i] = nums[i] + dp[dq[0]]
        while dq and dp[dq[-1]] <= dp[i]:
            dq.pop()
        dq.append(i)
    return dp[-1]

```

7.9 Pattern 6 summary

Problem	What the deque stores	Front pop	Back pop
Sliding Window Max	indices, decreasing values	index left the window	<code>nums[back] ≤ x</code>
Sliding Window Min	indices, increasing values	index left the window	<code>nums[back] ≥ x</code>
Shortest Subarray ≥ K	start indices, increasing prefix	window already reaches <code>k</code>	<code>prefix[back] ≥ prefix[i]</code>
Constrained Subsequence Sum	indices, decreasing <code>dp</code>	<code>dq[0] < i - k</code>	<code>dp[back] ≤ dp[i]</code>
Abs Diff ≤ Limit	two deques (max and min)	index <code>< left</code>	dominated
Jump Game VI	indices, decreasing <code>dp</code>	<code>dq[0] < i - k</code>	<code>dp[back] ≤ dp[i]</code>

Part 8 · Pattern 7 — Queue + HashMap / Frequency

Recognition. You need both the arrival order and some changing information about validity or frequency.

```
Queue    → "Who came first?"
HashMap  → "Is this still valid?"
```

8.1 First Non-Repeating Character in a Stream

Characters arrive one by one. After each arrival report the **first character that has appeared exactly once so far**. Stream `a a b c b` → `a # b b c`.

Derive it. The frequency map can tell us `a → 2, b → 1, c → 1` — but a map **does not preserve arrival order**. So we also need a queue.

What goes into the queue? Not "the unique characters" — **the characters in their arrival order**. The queue may hold `[a, b, c]` while the map says `a → 2`. The queue holds arrivals; **the map decides which of them are still valid**.

Stale elements. The front may no longer be valid. Look at the front repeatedly:

```
queue [a, b, c]    freq a:3 b:2 c:1
front a → freq 3 → invalid → remove
front b → freq 2 → invalid → remove
front c → freq 1 → valid   → this is the answer
```

The queue does not know what "valid" means — **the hash map provides that information**. We remove `b` not because 2 is bad in general, but because *this problem* asks for frequency 1.

char	freq	stale front removed	queue	first unique
a	a:1	-	[a]	a
a	a:2	a, a	[]	#
b	a:2, b:1	-	[b]	b
c	a:2, b:1, c:1	-	[b, c]	b
b	a:2, b:2, c:1	b	[c, b]	c

```

from collections import deque

def first_non_repeating(stream):
    """After every character, the first character that has appeared exactly once so far ('#' if
    none)."""
    q = deque()
    freq = {}
    out = []
    for ch in stream:
        freq[ch] = freq.get(ch, 0) + 1
        q.append(ch)
        while q and freq[q[0]] > 1:                # stale front: it stopped being unique
            q.popleft()
        out.append(q[0] if q else '#')
    return out

```

Complexity. Every stale entry is removed once → **O(n) amortized**, O(n) space. This is the same "each element leaves once" argument as the monotonic deque.

8.2 First Unique Character in a String (LeetCode 387)

When the whole string is given up front, two passes are simpler: count, then find the first character with count 1. (No queue needed — *use the queue when the data is a stream.*)

```

from collections import Counter

def first_uniq_char(s):
    """LeetCode 387 - the counting version (two passes) is simpler when the whole string is given up
    front."""
    freq = Counter(s)
    for i, ch in enumerate(s):
        if freq[ch] == 1:
            return i
    return -1

```

8.3 First Unique Number (LeetCode 1429, premium)

`showFirstUnique()` returns the first still-unique number in the queue; `add(value)` appends a number. Numbers may repeat.

The same design with integers. A small variation: **only the first arrival of each value is enqueued** (`if count == 1:` `q.append(value)`), so the queue never holds a duplicate; stale fronts are dropped in `showFirstUnique`.

```

from collections import deque, Counter

class FirstUnique:
    """LeetCode 1429. `add` may receive numbers repeatedly; showFirstUnique returns the first still-
    unique number."""

    def __init__(self, nums):
        self.q = deque()
        self.count = Counter()
        for x in nums:
            self.add(x)

    def showFirstUnique(self):
        while self.q and self.count[self.q[0]] > 1:
            self.q.popleft()
        return self.q[0] if self.q else -1

    def add(self, value):
        self.count[value] += 1
        if self.count[value] == 1:
            self.q.append(value)
            # only the first arrival is queued

```

(Number of Recent Calls, §2.2, is also this pattern: the queue is arrival order, and *the window is the validity condition*.)

8.4 Pattern 7 summary

Problem	Queue holds	Map holds	Front is valid when
First Non-Repeating Char (stream)	characters in arrival order	frequency	<code>freq[front] == 1</code>
First Unique Number	first arrivals	counts	<code>count[front] == 1</code>
Recent Calls	timestamps	(the time window)	<code>front ≥ t - 3000</code>

Part 9 · Pattern 8 — Queue Simulation

Recognition. Each entity gets a turn, performs some work, and may return to the queue. It overlaps with Pattern 1, but the emphasis is on **time, rounds and turns** — CPU scheduling, customers being served, round-robin processing.

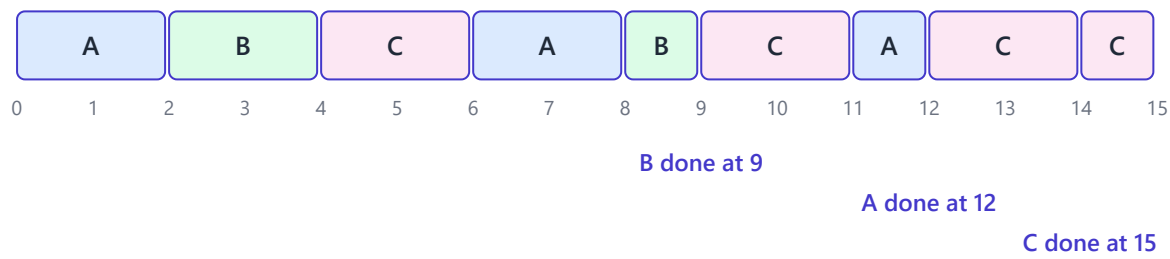
take → perform one unit of work → finished?
↓ no
back

Simulation invariant. The queue contains all unfinished tasks, in the order in which they should receive their next unit of processing.

9.1 CPU Round Robin

Tasks A = 5 , B = 3 , C = 7 all arrive at time 0. The CPU gives each task **at most quantum = 2** time units, then moves an unfinished task to the back. Report when each task completes.

Round robin, quantum 2: A=5, B=3, C=7 (each bar is one time slice)



task	time slice	ran	remaining	outcome	queue after
A	0 → 2	2	3	back	[B(3), C(7), A(3)]
B	2 → 4	2	1	back	[C(7), A(3), B(1)]
C	4 → 6	2	5	back	[A(3), B(1), C(5)]
A	6 → 8	2	1	back	[B(1), C(5), A(1)]
B	8 → 9	1	0	done at 9	[C(5), A(1)]
C	9 → 11	2	3	back	[A(1), C(3)]
A	11 → 12	1	0	done at 12	[C(3)]
C	12 → 14	2	1	back	[C(1)]
C	14 → 15	1	0	done at 15	[]

```

from collections import deque

def round_robin(tasks, quantum):
    """CPU round-robin. `tasks` = [(name, burst)] all arriving at time 0. Returns completion time
    per task."""
    q = deque(tasks)
    time = 0
    done = {}
    while q:
        name, remaining = q.popleft()
        run = min(quantum, remaining)           # a task never runs longer than it needs
        time += run
        remaining -= run
        if remaining == 0:
            done[name] = time
        else:
            q.append((name, remaining))
    return done

```

A task never runs longer than it needs: `run = min(quantum, remaining)`. The same simulation can return the **timeline** of slices (used for the Gantt chart above):

```

def round_robin_slices(tasks, quantum):
    """The same simulation, but returning the timeline [(task, start, end)] for a Gantt-style
    trace."""
    q = deque(tasks)
    time = 0
    timeline = []
    while q:
        name, remaining = q.popleft()
        run = min(quantum, remaining)
        timeline.append((name, time, time + run))
        time += run
        remaining -= run
        if remaining:
            q.append((name, remaining))
    return timeline

```

Complexity. Each iteration runs one slice; there are $\sum \lceil \text{burst} / \text{quantum} \rceil$ slices. Space $O(n)$.

Other quantities. *Turnaround time* = completion – arrival; *waiting time* = turnaround – burst. With arrival 0: A waits $12 - 5 = 7$, B waits $9 - 3 = 6$, C waits $15 - 7 = 8$.

9.2 Where else this pattern appears

- **Time Needed to Buy Tickets** (§2.1) and **Dota2 Senate** (§2.5) are the same "take, act, maybe return" simulation.

- **Correction.** The session lists "*Task Scheduler-style simulations*" under this pattern. **Task Scheduler (LeetCode 621)** is not a queue simulation: the cooling interval is solved with **greedy counting** or a **max-heap** (it is in the Heap guide). A queue *can* model the cooldown ("tasks that must wait `n` more slots"), but the heap is what chooses **which** task to run — a priority decision, not an arrival-order one.

9.3 Pattern 8 summary

Problem	The "work"	Returns to the queue when
Round Robin	<code>min(quantum, remaining)</code>	<code>remaining > 0</code>
Time Needed to Buy Tickets	buy 1 ticket	still needs tickets
Dota2 Senate	ban an opponent	always (at <code>+n</code>) — until a party is empty

Part 10 · Pattern 9 — Topological Sort (Kahn's Algorithm)

Recognition. *Dependencies / prerequisites and you need a valid ordering.* Words to listen for: *prerequisites, dependencies, course schedule, build order, task ordering, dependency resolution.*

10.1 Deriving it

Suppose

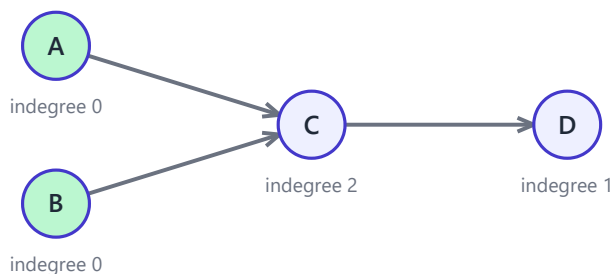
A → C (A must happen before C)
B → C
C → D

Which nodes can we process first, before anything else? Those with **no prerequisite**: A and B.

Indegree = the number of prerequisites a node still has.

Node	A	B	C	D
indegree	0	0	2	1

Kahn's algorithm: a node enters the queue when its indegree reaches 0



queue: [A, B]

process A → C: 2 → 1

process B → C: 1 → 0 ⇒ enqueue C

process C → D: 1 → 0 ⇒ enqueue D

order: A, B, C, D

cycle: A → B → C → A (every node keeps indegree 1, nothing ever enters the queue)

The algorithm.

1. Compute the indegree of every node.
2. Put **all** indegree-0 nodes into the queue.
3. Take a node from the queue.
4. "Complete" it: **decrease the indegree of every neighbour** (each has one fewer unfinished prerequisite).
5. If a neighbour's indegree reaches 0, add it to the queue.
6. Repeat.

Mental model. Processing a node removes one prerequisite from each of its dependents.

process	enqueued (indegree hit 0)	indegrees after	queue after
start	-	A=0, B=0, C=2, D=1	[A, B]
A	-	A=0, B=0, C=1, D=1	[B]

process	enqueued (indegree hit 0)	indegrees after	queue after
B	C	A=0, B=0, C=0, D=1	[C]
C	D	A=0, B=0, C=0, D=0	[D]
D	-	A=0, B=0, C=0, D=0	[]

Order: **A** → **B** → **C** → **D**, all 4 nodes processed → no cycle.

Invariant. A node enters the queue exactly when all of its prerequisites have been processed.

10.2 Cycle detection

If some nodes never reach indegree 0, **no valid ordering can include every node** — the graph has a cycle.

A → B → C → B indegrees: A: 0 B: 2 C: 1

process	enqueued (indegree hit 0)	indegrees after	queue after
start	-	A=0, B=2, C=1	[A]
A	-	A=0, B=1, C=1	[]

Only **1 of 3** nodes was processed and the queue is empty: **B** and **C** wait for each other forever → cycle.

The detection rule.

If `processed_count < number_of_nodes`, a cycle exists. If `processed_count == number_of_nodes`, every node was ordered — no cycle.

"But a node might be unreachable without being in a cycle?"

Good question — and the answer is why we initialise the queue with **every** indegree-0 node, not just one:

A → B C
indegrees: A: 0, B: 1, C: 0 queue starts as [A, C]

A is processed (B → 0), **C** is processed, **B** is processed: all 3 nodes done, **no cycle**. **C** being disconnected from **A** → **B** is harmless.

Disconnected ≠ cyclic. Kahn's algorithm is not asking "is everything reachable from one source?". If `processed_count < n`, some nodes' prerequisites can *never* all be satisfied — and in a finite directed graph that means a **directed cycle**.

10.3 The template

```
from collections import deque

def topo_order(n, edges):
    """Kahn's algorithm. edges = [(u, v)] meaning u must come BEFORE v. Returns an order, or None
    when there is a cycle."""
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1
    q = deque(i for i in range(n) if indegree[i] == 0)    # every node with no prerequisite, not
just one
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nb in graph[node]:
            indegree[nb] -= 1                                # one prerequisite of nb is now done
            if indegree[nb] == 0:
                q.append(nb)                                # ALL prerequisites done: nb may start
    return order if len(order) == n else None
```

Complexity. $O(V + E)$ time and space — each node is dequeued once, each edge relaxed once.

10.4 Course Schedule (LeetCode 207) and Course Schedule II (LeetCode 210)

`prerequisites[i] = [a, b]` means *take b before a*. **207:** can you finish all courses? **210:** return a valid order (or []).

Note the edge direction: `[a, b]` is the edge `b → a`. 207 is "is the graph acyclic?"; 210 is "give me the order Kahn produced."

```
def can_finish(numCourses, prerequisites):
    """LeetCode 207: can all courses be taken? (is the prerequisite graph acyclic?) prerequisites =
[[course, prereq]]"""
    return topo_order(numCourses, [(p, c) for c, p in prerequisites]) is not None
```

```
def find_order(numCourses, prerequisites):
    """LeetCode 210: one valid order, or [] if impossible."""
    order = topo_order(numCourses, [(p, c) for c, p in prerequisites])
    return order if order is not None else []
```

Example: `[[1,0],[2,0],[3,1],[3,2]] → [0, 1, 2, 3]` (or `[0, 2, 1, 3]`; any valid order is accepted).

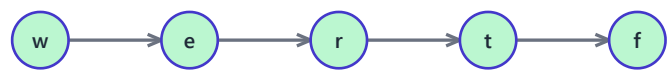
10.5 Alien Dictionary (LeetCode 269)

Words are sorted according to an unknown alphabet: ["wrt", "wrf", "er", "ett", "rftt"] . Derive the order of the letters.

The graph is not given — it hides in the **order of adjacent words**. Compare each adjacent pair; the **first differing letter** gives one fact ("x comes before y"); later letters say nothing.

Alien Dictionary: compare ADJACENT words, the first difference gives an edge

wrt	wrt / wrf	t -> f
wrf	wrf / er	w -> e
er	er / ett	r -> t
ett	ett / rftt	e -> r
rftt		



topological order: w e r t f

a longer word before its own prefix ("abc" before "ab") is impossible -> return ""

adjacent words	first difference	edge (x before y)
wrt, wrf	t ≠ f	t → f
wrf, er	w ≠ e	w → e
er, ett	r ≠ t	r → t
ett, rftt	e ≠ r	e → r

Kahn on this graph gives **wertf** .

Subtleties.

- Only the **first** differing character matters.
- "abc" followed by "ab" is **invalid** (a word cannot come before its own prefix) → return "" .
- Every letter appears in the graph, even with no edges — and any valid topological order is accepted.
- A cycle (["z", "x", "z"]) → "" .

```

from collections import deque

def alien_order(words):
    """LeetCode 269. The dependency graph is hidden in the order of ADJACENT words."""
    graph = {ch: set() for w in words for ch in w}
    indegree = {ch: 0 for ch in graph}
    for a, b in zip(words, words[1:]):
        if len(a) > len(b) and a.startswith(b):
            return "" # "abc" before "ab" is impossible
        for x, y in zip(a, b):
            if x != y: # only the FIRST difference is a fact
                if y not in graph[x]:
                    graph[x].add(y)
                    indegree[y] += 1
                break
    q = deque(ch for ch in indegree if indegree[ch] == 0)
    order = []
    while q:
        ch = q.popleft()
        order.append(ch)
        for nb in graph[ch]:
            indegree[nb] -= 1
            if indegree[nb] == 0:
                q.append(nb)
    return "".join(order) if len(order) == len(graph) else ""

```

Complexity. With C the total number of characters in all words: $O(C)$ time and space (the alphabet is bounded).

10.6 Parallel Courses (LeetCode 1136, premium)

n courses, $\text{relations}[i] = [\text{prev}, \text{next}]$. Each **semester** you may take any number of courses whose prerequisites were finished in an earlier semester. Minimum number of semesters, or -1 if impossible.

Kahn **by levels**: each level of the queue (frozen with $\text{len}(q)$, exactly like BFS level order) is one semester. The answer is the number of levels — the length of the **longest dependency chain**.

```

from collections import deque

def minimum_semesters(n, relations):
    """LeetCode 1136 - the number of Kahn LEVELS (each level = one semester), or -1 if there is a
    cycle.
    relations = [[prev, next]] with courses numbered 1..n."""
    graph = [[] for _ in range(n + 1)]
    indegree = [0] * (n + 1)
    for u, v in relations:
        graph[u].append(v)
        indegree[v] += 1
    q = deque(i for i in range(1, n + 1) if indegree[i] == 0)
    semesters = taken = 0
    while q:
        semesters += 1
        for _ in range(len(q)):
            this semester
            node = q.popleft()
            taken += 1
            for nb in graph[node]:
                indegree[nb] -= 1
                if indegree[nb] == 0:
                    q.append(nb)
    return semesters if taken == n else -1

```

everything available now can be taken

10.7 Pattern 9 summary

Problem	Nodes	Edge $u \rightarrow v$ means	Answer
Course Schedule	courses	<code>u</code> is a prerequisite of <code>v</code>	acyclic?
Course Schedule II	courses	same	the order
Alien Dictionary	letters	<code>u</code> sorts before <code>v</code>	the order (or <code>""</code>)
Parallel Courses	courses	same	number of Kahn levels

Part 11 · Pattern 10 — 0–1 BFS

Recognition. Shortest path where every edge has weight 0 or 1.

all edges weigh 1 → BFS
edge weights are 0 or 1 → 0–1 BFS
arbitrary non-negative → Dijkstra

11.1 Why a plain queue no longer works

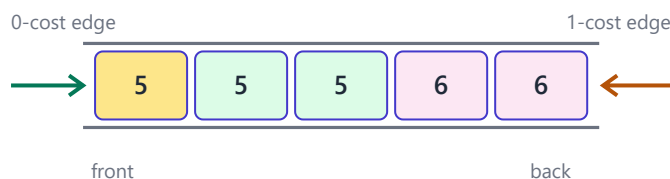
Normal BFS works because every edge costs exactly 1, so a FIFO queue processes nodes in increasing distance. With weights 0 and 1 and current distance 5:

move with cost 0 → distance 5 (as close as the node we are processing)
move with cost 1 → distance 6

The cheaper node should be processed **first**. So use a **deque**:

edge cost 0 → add to the FRONT
edge cost 1 → add to the BACK

0-1 BFS: the deque holds only two distances, d (front block) and $d+1$ (back block)



pop a node with dist 5:

edge cost 0 → dist 5 → push FRONT (it is as close as the node we are processing)

edge cost 1 → dist 6 → push BACK (it belongs behind every distance-5 node)

(the chat's picture [5 5 5 6 6 7 7] cannot occur: max - min of the deque never exceeds 1)

11.2 Why front and back? (it is not arbitrary)

Suppose the deque is $[A(5), B(5)]$ and we process A , with $A \xrightarrow{0} C$ and $A \xrightarrow{1} D$:

$C = 5 + 0 = 5$ → same distance as A → push FRONT (process it as soon as possible)
 $D = 5 + 1 = 6$ → one level farther away → push BACK (it belongs behind every distance-5 node)
result: $[C(5), B(5), D(6)]$

With ordinary BFS every edge goes to the next level, so one queue end is enough. With 0–1 BFS the two kinds of edge lead to **two different levels** (d and $d + 1$), and the deque is a very light-weight **priority queue that only has to distinguish "same distance" from "one more"**.

11.3 The invariant (and a correction)

The deque holds nodes in non-decreasing distance, and the distances in it never differ by more than 1 — it looks like $[d, d, d, d+1, d+1]$.

Correction. The session pictured the deque as $[5, 5, 5, 6, 6, 7, 7]$. That can never happen: the front block is at distance d (the node just popped) and everything pushed to the back is $d + 1$; a $d + 2$ would need a node at $d + 1$ to have been processed, which happens only after all the d s are gone. The tests behind this guide record the distance of every entry in the deque and confirm **$\max - \min \leq 1$ at every moment** on thousands of random graphs.

11.4 Relaxation

If a shorter route to a node is found, **update it**:

`dist[node] = 10 ; a new path gives 8 → dist[node] = 8, and push it to the end that matches the edge cost`

Relaxation rule. if `new_dist < dist[neighbor]`: update it and add the neighbour to the deque. This is the connection between 0–1 BFS and **Dijkstra**: 0–1 BFS is Dijkstra whose priority queue collapsed into a deque.

A node may enter the deque more than once (each time its distance improves). That is correct; the improved distance simply wins. The total work is still $O(V + E)$.

Graph: $0 \rightarrow 1$, $0 \rightarrow 1 \rightarrow 2$, $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$.

dequeue	relaxations	deque after	dist
start	-	[0]	[0, ∞, ∞, ∞]
0	1 (dist 0) → FRONT, 2 (dist 1) → BACK	[1, 2]	[0, 0, 1, ∞]
1	3 (dist 0) → FRONT	[3, 2]	[0, 0, 1, 0]
3	-	[2]	[0, 0, 1, 0]
2	-	[]	[0, 0, 1, 0]

```

from collections import deque

def zero_one_bfs(graph, start):
    """graph[u] = [(v, w)] with w in {0, 1}. Cost-0 edges go to the FRONT of the deque, cost-1 edges
    to the BACK."""
    INF = float('inf')
    dist = [INF] * len(graph)
    dist[start] = 0
    dq = deque([start])
    while dq:
        node = dq.popleft()
        for nb, w in graph[node]:
            nd = dist[node] + w
            if nd < dist[nb]:
                # relaxation: a strictly shorter path was
                found
                dist[nb] = nd
                if w == 0:
                    dq.appendleft(nb)
                else:
                    dq.append(nb)
    return dist

```

Complexity. $O(V + E)$ time, $O(V)$ space — every relaxation is $O(1)$. (Dijkstra needs $O((V + E) \log V)$.)

11.5 Minimum Cost to Make at Least One Valid Path in a Grid (LeetCode 1368)

Each cell holds an arrow: 1 right, 2 left, 3 down, 4 up. You start at the top-left and must reach the bottom-right by following arrows. **Following** a cell's arrow costs 0; **changing** it costs 1. Minimum total cost.

Ask: *what does moving from one cell to another cost?* Either 0 (the arrow already points there) or 1 — the recognition signal. From `dist[r][c]` following the arrow gives `dist + 0` → **front**; any other direction gives `dist + 1` → **back**.

arrows in the row	cheapest cost to reach each cell of the row
→ → → →	0 0 0 0
← ← ← ←	1 1 1 1
→ → → →	2 2 2 2
← ← ← ←	3 3 3 3

The bottom-right cell costs 3: follow the free arrows along a row, then pay 1 to turn down at the end of each row.

```

from collections import deque

def min_cost_valid_path(grid):
    """LeetCode 1368. Arrows: 1 right, 2 left, 3 down, 4 up. Following the arrow costs 0, changing
    it costs 1."""
    rows, cols = len(grid), len(grid[0])
    INF = float('inf')
    dist = [[INF] * cols for _ in range(rows)]
    dist[0][0] = 0
    dq = deque([(0, 0)])
    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)] # index + 1 is the arrow number
    while dq:
        r, c = dq.popleft()
        for i, (dr, dc) in enumerate(dirs):
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            cost = 0 if grid[r][c] == i + 1 else 1
            nd = dist[r][c] + cost
            if nd < dist[nr][nc]:
                dist[nr][nc] = nd
                if cost == 0:
                    dq.appendleft((nr, nc))
                else:
                    dq.append((nr, nc))
    return dist[rows - 1][cols - 1]

```

Complexity. $O(\text{rows} \cdot \text{cols})$: $V = \text{rows} \times \text{cols}$, $E \approx 4V$.

This is probably the cleanest problem for understanding *why* we push to the front or the back.

11.6 Minimum Obstacle Removal to Reach the Corner (LeetCode 2290)

0 empty, 1 obstacle. Moving **into** an empty cell costs 0; moving into an obstacle costs 1 (remove it). Minimum removals from the top-left to the bottom-right.

edge cost $\in \{0, 1\}$ again: 0 $\rightarrow$ appendleft(), 1 $\rightarrow$ append(). The cost of an edge is simply the **value of the cell you step into**.

```

from collections import deque

def minimum_obstacles(grid):
    """LeetCode 2290. Entering an empty cell costs 0, entering an obstacle costs 1 (remove it)."""
    rows, cols = len(grid), len(grid[0])
    INF = float('inf')
    dist = [[INF] * cols for _ in range(rows)]
    dist[0][0] = 0
    dq = deque([(0, 0)])
    while dq:
        r, c = dq.popleft()
        for dr, dc in DIRS4:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                nd = dist[r][c] + grid[nr][nc]
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    if grid[nr][nc] == 0:
                        dq.appendleft((nr, nc))
                    else:
                        dq.append((nr, nc))
    return dist[rows - 1][cols - 1]

```

A good test of whether you understood 0–1 BFS rather than memorised the template: compare it with §5.4 (*at most k removals* is a **BFS with state**; *fewest removals* is a **0–1 BFS**). Same grid, different question.

11.7 Pattern 10 summary

Problem	Edge cost	0 goes to	1 goes to
Min Cost Valid Path	0 if you follow the arrow, else 1	front	back
Minimum Obstacle Removal	value of the entered cell	front	back
Generic weighted graph	given 0 / 1	front	back

Part 12 · Templates, Decision Guide and Cheat Sheets

12.1 The templates to memorise

Template 1 — Basic FIFO simulation

```
from collections import deque
q = deque(initial_items)
while q:
    item = q.popleft()
    # process item
    if not_finished:
        q.append(item)
```

Mental model: FRONT → process → BACK. *Problems:* Time Needed to Buy Tickets, Dota2 Senate, Students Unable to Eat Lunch.

Template 2 — BFS / level order

```
from collections import deque
q = deque([start])
visited = {start}
while q:
    node = q.popleft()
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)          # mark on ENQUEUE
            q.append(neighbor)

# level by level
while q:
    level_size = len(q)                  # freeze the boundary
    for _ in range(level_size):
        node = q.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                q.append(neighbor)
```

Invariant: the queue processes nodes in **non-decreasing distance** from the source.

Template 3 — Multi-source BFS

```
q = deque()
for source in sources:
    q.append(source); visited[source] = True; distance[source] = 0
while q:
    node = q.popleft()
    for neighbor in graph[node]:
        if not visited[neighbor]:
            visited[neighbor] = True
            distance[neighbor] = distance[node] + 1
            q.append(neighbor)
```

Template 4 — BFS + state

```
q = deque([(start, initial_state)])
visited = {(start, initial_state)}
while q:
    position, state = q.popleft()
    for next_position, next_state in get_next_states(position, state):
        if (next_position, next_state) not in visited:
            visited.add((next_position, next_state))
            q.append((next_position, next_state))
```

Key question: what information must travel with me because it changes my future choices?

Template 5 — BFS on a grid

```
DIRS4 = [(-1, 0), (1, 0), (0, -1), (0, 1)]
q = deque([(sr, sc, 0)]) # (row, col, distance)
visited = {(sr, sc)}
while q:
    r, c, d = q.popleft()
    for dr, dc in DIRS4:
        nr, nc = r + dr, c + dc
        if not (0 <= nr < rows and 0 <= nc < cols): continue
        if grid[nr][nc] is invalid or (nr, nc) in visited: continue
        visited.add((nr, nc))
        q.append((nr, nc, d + 1))
```

Template 6 — Monotonic deque (window maximum)

```
dq = deque()
for i, x in enumerate(nums):
    while dq and dq[0] <= i - k:          # front: EXPIRED
        dq.popleft()
    while dq and nums[dq[-1]] <= x:      # back: DOMINATED
        dq.pop()
    dq.append(i)
    if i >= k - 1:
        answer.append(nums[dq[0]])
```

For a minimum: flip the comparison. Store indices.

Template 7 — Queue + hash map

```
q, freq = deque(), Counter()
for x in stream:
    freq[x] += 1
    q.append(x)
    while q and not valid(q[0], freq):    # drop STALE fronts
        q.popleft()
    answer = q[0] if q else default
```

Queue = "who came first?" · Map = "is it still valid?"

Template 8 — Queue simulation

```
q = deque(tasks)
while q:
    task = q.popleft()
    task.work -= min(quantum, task.work)
    if task.work > 0:
        q.append(task)
```

Template 9 — Topological sort (Kahn)

```
indegree = [0] * n
for u in range(n):
    for v in graph[u]:
        indegree[v] += 1
q = deque(i for i in range(n) if indegree[i] == 0)
order = []
while q:
    node = q.popleft()
    order.append(node)
    for nb in graph[node]:
        indegree[nb] -= 1
        if indegree[nb] == 0:
            q.append(nb)
if len(order) != n:
    ... # a cycle exists
```

Template 10 — 0–1 BFS

```
INF = float('inf')
dist = [INF] * n
dist[start] = 0
dq = deque([start])
while dq:
    node = dq.popleft()
    for nb, w in graph[node]:
        nd = dist[node] + w
        if nd < dist[nb]:
            dist[nb] = nd
            dq.appendleft(nb) if w == 0 else dq.append(nb)
```

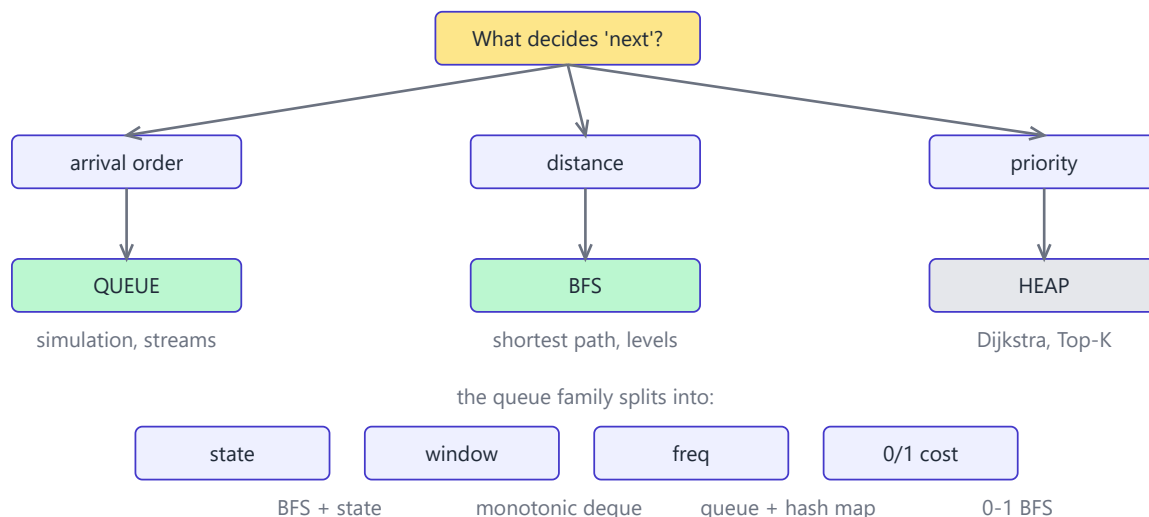
12.2 Final decision guide

The problem says...	Think...
Process things in arrival order	FIFO queue
Shortest number of steps , equal cost	BFS
Multiple starting points spreading at once	multi-source BFS
Position plus an extra condition (keys, fuel, walls broken)	BFS + state
Grid + shortest moves	grid BFS
Max / min of every window	monotonic deque
Order plus frequency / changing validity	queue + hash map
Repeated turns / rounds / time slices	queue simulation

The problem says...	Think...
Prerequisites / dependencies / valid ordering	Kahn's algorithm
Shortest path with edge weights 0 or 1	0-1 BFS

The deeper picture:

The deeper picture: what determines the NEXT element?



The hardest step is almost always identifying **what the queue element / state represents**. Once that is correct, the implementation tends to become mechanical.

12.3 The ten hard problems, one per pattern

#	Pattern	Problem	The insight
1	FIFO	Dota2 Senate (649)	queue turn numbers ; the smaller acts first and returns at +n
2	BFS	Word Ladder (127)	the graph is implicit: word = node, one-letter change = edge
3	Multi-source	01 Matrix (542)	all 0 s are the initial queue — don't search from each 1
4	BFS + state	Obstacles Elimination (1293)	state = (row, col, eliminated)
5	Grid BFS	Escape a Large Maze (1036)	bound the search by $B(B-1)/2$, run from both ends
6	Monotonic deque	Constrained Subsequence Sum (1425)	DP + sliding-window max over dp
7	Queue + map	First Non-Repeating in a Stream	queue = order, map = validity, drop stale fronts
8	Simulation	CPU Round Robin	take → run a quantum → finished? else back
9	Topological	Alien Dictionary (269)	build the graph from adjacent words, then Kahn
10	0-1 BFS	Min Cost Valid Path (1368)	following the arrow = 0 → front, changing it = 1 → back

12.4 Complexity cheat sheet

Algorithm	Time	Space	Note
Queue operations (<code>deque</code>)	$O(1)$ each	—	<code>list.pop(0)</code> is $O(n)$
BFS (graph)	$O(V + E)$	$O(V)$	each node enqueued once
BFS on an $R \times C$ grid	$O(R \cdot C)$	$O(R \cdot C)$	$\times 4$ (or $\times 8$) neighbour checks
BFS + state with S states per cell	$O(R \cdot C \cdot S)$	$O(R \cdot C \cdot S)$	e.g. $k + 1$ layers, 2^{keys} masks
Multi-source BFS	$O(V + E)$	$O(V)$	same as BFS
Word Ladder	$O(26 \cdot N \cdot L^2)$	$O(N \cdot L)$	bidirectional BFS is much faster in practice
Monotonic deque	$O(n)$ amortized	$O(k)$	each index in once, out once
Queue + hash map (stream)	$O(1)$ amortized per item	$O(n)$	stale entries removed once
Round robin	$O(\text{number of slices})$	$O(n)$	
Kahn	$O(V + E)$	$O(V + E)$	
0–1 BFS	$O(V + E)$	$O(V)$	Dijkstra is $O((V + E) \log V)$

12.5 Pitfalls checklist

Pitfall	Fix
<code>list.pop(0)</code> as a queue	<code>collections.deque</code>
Marking visited on dequeue	mark when you enqueue (§6.2)
Missing visited on a cyclic graph	infinite loop
Marking visited on only one branch (the wall branch of §5.3)	test visited in every branch that enqueues
Level loop written <code>while q</code> inside the level	<code>for _ in range(len(q))</code> — freeze the size first
Visited keyed by position only, while extra information matters	key it by the whole state
Returning distance at the wrong moment (counting cells vs edges)	decide whether the start counts as 0 or 1
Monotonic deque storing values, not indices	you cannot tell when an entry has expired
Popping the deque's back while <code><</code> should be <code><=</code> (or vice versa)	decide how to treat ties deliberately (§7.3)
Kahn started from a single source	start from all indegree-0 nodes
Kahn edge direction reversed (<code>[a, b]</code> means <code>b → a</code>)	draw one example first
0–1 BFS pushing to the wrong end	cost 0 → front, cost 1 → back
Mutating the input grid when you should not	use a separate visited
Forgetting a node may enter the 0–1 deque twice	the relaxation check <code>nd < dist[nb]</code> handles it
Using BFS for weighted shortest paths	0–1 BFS (weights 0/1) or Dijkstra

12.6 Which patterns combine?

- **Multi-source BFS + grid** — Rotting Oranges, Walls and Gates.
 - **Grid BFS + state** — Obstacles Elimination, All Keys.
 - **Kahn + levels** — Parallel Courses (each level = a semester).
 - **DP + monotonic deque** — Constrained Subsequence Sum, Jump Game VI.
 - **Sliding window + two monotonic deques** — Abs Diff $\leq$ Limit.
 - **Queue + hash map + window** — Number of Recent Calls, Hit Counter.
 - **0–1 BFS vs BFS + state** — *fewest* obstacle removals (0–1) vs *at most k* removals (state).
 - **Queue vs stack** — Queue from two stacks (232), Stack from queues (225); monotonic **stack** (Stack guide) vs monotonic **deque** (this guide).
-

Part 13 · Practice, Recognition Drills and Interview Q&A

13.1 Practice list (LeetCode)

Pattern	Start with	Then	Hard
FIFO simulation	Number of Recent Calls (933) · Moving Average (346) · Students Unable to Eat (1700)	Time Needed to Buy Tickets (2073) · Design Circular Queue (622) · Hit Counter (362)	Dota2 Senate (649) · Reveal Cards (950)
BFS / level order	Binary Tree Level Order (102) · Minimum Depth (111) · Average of Levels (637)	Right Side View (199) · Zigzag (103) · Open the Lock (752) · Perfect Squares (279)	Word Ladder (127) · Snakes and Ladders (909)
Multi-source BFS	Rotting Oranges (994)	01 Matrix (542) · Walls and Gates (286)	As Far from Land (1162)
BFS + state	Minimum Genetic Mutation (433)	Obstacles Elimination (1293) · Minimum Knight Moves (1197)	Shortest Path to Get All Keys (864)
Grid BFS	Number of Islands (200) · Nearest Exit (1926)	Shortest Path in Binary Matrix (1091)	Escape a Large Maze (1036)
Monotonic deque	Sliding Window Maximum (239)	Abs Diff $\leq$ Limit (1438) · Jump Game VI (1696)	Constrained Subsequence Sum (1425) · Shortest Subarray with Sum $\geq K$ (862)
Queue + hash map	First Unique Character (387)	First Unique Number (1429)	(stream design variants)
Simulation	Time Needed to Buy Tickets (2073)	Round-robin scheduling	—
Topological sort	Course Schedule (207)	Course Schedule II (210) · Parallel Courses (1136)	Alien Dictionary (269)
0-1 BFS	Minimum Obstacle Removal (2290)	Min Cost to Make a Valid Path (1368)	—

13.2 Recognition drills

For each statement, decide *pattern* $\rightarrow$ *what one queue element is* $\rightarrow$ *the invariant* **before** reading the answers.

#	Statement	Answer
1	"Return the number of requests made in the last 5 minutes."	Pattern 1/7 · a timestamp · the deque holds exactly the timestamps in the window
2	"Minimum number of button presses to turn <code>start</code> into <code>target</code> ."	BFS · a state (value) · nearer states first
3	"Time until every cell is infected / every orange has rotted."	Multi-source BFS · a currently infected cell · each level is one time unit
4	"Fewest moves, but you have 3 teleports / can break 2 walls."	BFS + state · (<code>cell</code> , <code>resources left</code>) · visited keyed by the full state

#	Statement	Answer
5	"Maximum of every window of size k."	Monotonic deque · an index · decreasing values, front = max
6	"First unique element seen so far, after each arrival."	Queue + hash map · an arrival · the map decides validity
7	"Given prerequisites, is there a valid order?"	Kahn · a node with no unfinished prerequisites · enters when its indegree hits 0
8	"Moving costs 0 if the arrow points that way, else 1."	0–1 BFS · a cell · the deque holds distances <code>d</code> and <code>d+1</code> only
9	"Each process gets 4 ms, then goes to the back."	Simulation · a task · the front gets the CPU next
10	"Cheapest route with arbitrary non-negative prices."	Not a queue — Dijkstra with a heap.

13.3 Interview questions and short answers

Why use a deque instead of a list for a queue? `list.pop(0)` is $O(n)$ because every remaining element shifts; `deque.popleft()` is $O(1)$.

When does BFS give the shortest path? When every edge has the **same cost**. The first time BFS reaches a node it does so along a path with the fewest edges, because nodes are processed in non-decreasing distance.

Why mark a node visited when enqueueing rather than dequeuing? Otherwise several nodes can discover it before it is dequeued and it enters the queue multiple times. Marking on enqueue keeps the invariant: *a state in the queue is already visited, exactly once*.

How do you separate levels in BFS? Record `level_size = len(queue)` at the start of each level and process exactly that many nodes.

What is multi-source BFS and when is it better? Put *all* sources into the queue at distance 0. It finds each node's distance to its nearest source in **one** pass instead of one BFS per node.

What is "BFS + state"? When the future depends on extra information (walls broken, keys held), that information is part of the node. `visited` is keyed by the whole state.

Why does the monotonic deque run in $O(n)$ despite the inner `while`? Each index is pushed once and popped at most once (from the front *or* the back), so the total pops $\leq n$ — amortized $O(1)$ per element.

Why store indices in a monotonic deque? Values say whether an element is *useful*; indices say whether it is still *inside the window*.

Explain Kahn's algorithm and cycle detection. Repeatedly take a node with indegree 0, output it, and decrement its neighbours' indegrees. If the output has fewer than `n` nodes, the rest sit on (or behind) a cycle.

Why does 0–1 BFS use a deque? Cost-0 edges lead to the *same* distance level (push front), cost-1 edges to the *next* (push back); the deque keeps distances in non-decreasing order without a heap.

BFS vs DFS vs Dijkstra? BFS — equal-cost edges, queue. DFS — reachability / exhaustive search, stack. 0–1 BFS — weights 0/1, deque. Dijkstra — arbitrary non-negative weights, heap.

How can a queue be built from two stacks (or a stack from queues)? Two stacks: `inbox` for pushes, `outbox` for pops, pour when `outbox` is empty (amortized $O(1)$). One queue as a stack: after each push, rotate the older elements behind the new one.

13.4 One-page recap

QUEUE first in, first out. deque: $O(1)$ at both ends. `list.pop(0)` is $O(n)$.
AMORTIZED each element enters once and leaves once → the inner while is $O(1)$ per element.

WHAT DETERMINES "NEXT"?

arrival order → queue distance → BFS priority → heap

PATTERNS

1 FIFO simulation	the front is whoever is next; unfinished ones go back
2 BFS / levels	mark on ENQUEUE; <code>level_size = len(q)</code> freezes the level
3 multi-source BFS	all sources in the queue at distance 0
4 BFS + state	(position, extra info); visited keyed by the whole state
5 grid BFS	4 / 8 neighbours; inside? allowed? unvisited?
6 monotonic deque	indices; front = EXPIRED, back = DOMINATED; front = the extreme
7 queue + hash map	queue = order, map = validity; drop stale fronts
8 simulation	take → work → finished? else back
9 Kahn	all indegree-0 nodes first; processed < n → cycle
10 0-1 BFS	cost 0 → front, cost 1 → back; deque holds only d and d+1

ASK what is one queue element? what is the invariant? when do I mark it seen?